

Introduzione

Una introduzione alla tecnologia del video digitale, destinata prevalentemente a ingegneri o sviluppatori software, ma sufficientemente **semplice per consentire a chiunque di imparare**. L'idea è nata durante un [workshop sulla tecnologia video per principianti](#).

L'obiettivo è introdurre alcuni importanti concetti sul video digitale, usando un **vocabolario semplice, molte figure e esempi pratici** quando possibile, e rendere questa conoscenza disponibile ovunque. Non esitare a inviare correzioni o suggerimenti!

Ci saranno delle sezioni con esempi e esercizi che richiedono di avere Docker installato e una copia di questa repository scaricata in locale.

```
git clone https://github.com/leandromoreira/digital_video_introduction.git
cd digital_video_introduction
./setup.sh
```

ATTENZION : quando vedi scritto un comando come `./s/ffmpeg` oppure `./s/mediainfo`, significa che si sta eseguendo quel programma all'interno di un container Docker, che contiene tutte le dipendenze richieste per funzionare.

Tutti gli esempi pratici devono essere eseguiti dalla repository locale che hai scarica . Per gli esempi di jupyter, devi avviare il server eseguendo `./s/start_jupyter.sh`, e aprire in un browser l'URL mostrato.

Indice

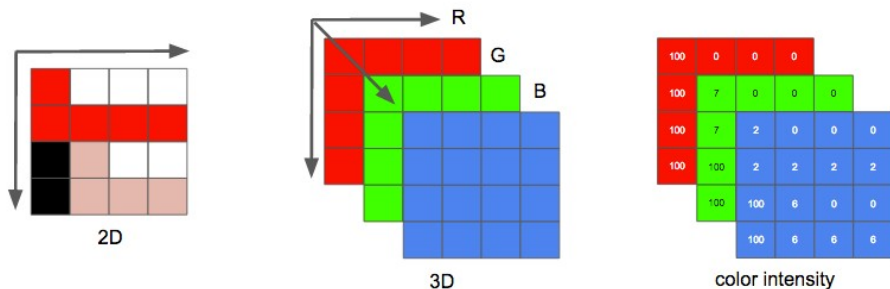
- [Introduzione](#)
- [Indice](#)
- [Terminologia](#)
 - [Altri modi per codificare un'immagine a colori](#)
 - [Esercizio: sperimentare con le immagini e i colori](#)
 - [I DVD hanno un DAR 4:3](#)
 - [Esercizio: controllare le proprietà di un file video](#)
- [Rimozione delle informazioni ridondanti](#)
 - [I colori, la luminanza e i nostri occhi](#)
 - [Modelli di colore](#)
 - [Conversione tra YCbCr e RGB](#)
 - [Sottocampionamento della cromaticità](#)
 - [Esercizio: vedere l'istogramma YCbCr](#)
 - [Tipi di fotogramma](#)
 - [I Frame \(intra, keyframe\)](#)

- [P Frame \(predicted\)](#)
 - [Esercizio: un video con un solo I-frame](#)
- [B Frame \(bi-predictive\)](#)
 - [Esercizio: confronta video con e senza B-frame](#)
- [Sommaro](#)
- [Ridondanza temporale \(predizione inter-frame\)](#)
 - [Esercizio: vedere i vettori di movimento](#)
- [Ridondanza spaziale \(predizione intra-frame\)](#)
 - [Esercizio: vedere la predizione intra-frame](#)
- [Come funziona un codec video?](#)
 - [Cosa? Perché? Come?](#)
 - [Storia](#)
 - [La nascita di AV1](#)
 - [Un codec generico](#)
 - [1° passo - partizionamento dell'immagine](#)
 - [Esercizio: vedere le partizioni](#)
 - [2° passo - predizioni](#)
 - [3° passo - trasformazione](#)
 - [Esercizio: scartare diversi coefficienti](#)
 - [4° passo - quantizzazione](#)
 - [Esercizio: quantizzazione](#)
 - [5° passo - codifica dell'entropia](#)
 - [Codifica VLC](#)
 - [Codifica aritmetica](#)
 - [Esercizio: CABAC vs CAVLC](#)
 - [6° passo - formato bitstream](#)
 - [Bitstream di H.264](#)
 - [Esercizio: ispezionare il bitstream H.264](#)
 - [Ripasso](#)
 - [Come fa H.265 a comprimere più di H.264?](#)
- [Streaming online](#)
 - [Architettura generale](#)
 - [Download progressivo e download adattivo](#)
 - [Protezione dei contenuti](#)
- [Come usare jupyter](#)
- [Conferenze](#)
- [Riferimenti](#)

Terminologia di base

Un'immagine può essere pensata come una **matrice bidimensionale**. Se pensiamo al fatto che ci sono anche i **color**, possiamo evolvere questa idea pensando l'immagine come una **matrice tridimensionale**, dove la **terza dimensione** (o le dimensioni aggiuntive) viene utilizzata per memorizzare le **informazioni sul color**.

Se scegliamo di rappresentare i colori utilizzando i **tre colori primari (rosso, verde e blu)**, possiamo definire tre piani: uno per il **rosso**, uno per il **ver** e uno per il **blu**.



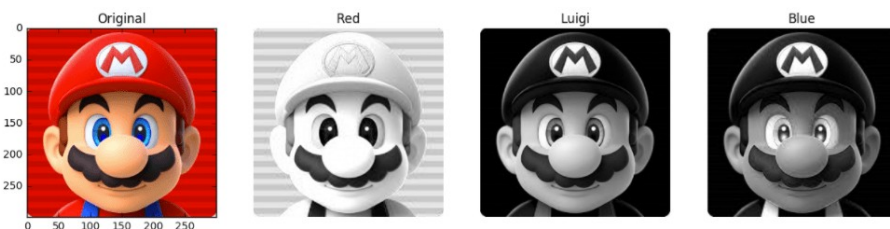
Chiameremo ogni punto di questa matrice un **pixel** (dall'inglese *picture element*). Un pixel rappresenta l'**intensità** (che in genere è un valore numerico) di un determinato colore. Ad esempio, un **pixel rosso** può essere rappresentato con zero verde, zero blu e il massimo rosso possibile. Un **pixel di colore rosa** può essere formato con una combinazione di tre colori: usando una rappresentazione numerica in un intervallo che va da 0 a 255, il rosa può essere definito come **Rosso=255, Verde=192 e Blu=203**.

Altri modi per codificare un'immagine a colori

Si possono utilizzare molti altri modelli per rappresentare i colori che compongono un'immagine. Potremmo ad esempio utilizzare una tavolozza di colori (con un indice associato a ciascun colore), rappresentando poi i pixel colorati con l'indice del colore nella tavolozza. In questo modo ogni pixel richiederebbe soltanto un byte di memoria, anziché tre byte come nel caso del modello RGB. Il vantaggio di un modello a *palette* è quello di occupare meno memoria, riducendo però il numero di colori a disposizione.

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F
30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F

Per fare un esempio, guarda le immagini qui sotto. La prima figura è completamente colorata. Le altre mostrano rispettivamente i piani (o canali) rosso, verde e blu, in scala di grigi.

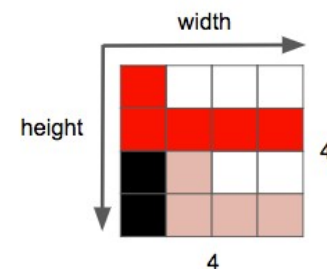


Possiamo notare che il **colore rosso** è quello che **contribuisce di più** a creare il colore finale (si notino le parti più chiare nella seconda figura, sotto "Red"), mentre il **colore blu** contribuisce prevalentemente a rendere il colore degli **occhi di Mario** e parte dei suoi vestiti (ultima figura a destra). Si può anche notare come nessun canale RGB dia un forte contributo a realizzare i colori scuri, come i baffi di Mario.

Ogni colore richiede un certo numero di bit per essere rappresentato. Questo valore si chiama **profondità di bit**. Se ad esempio decidiamo di utilizzare **8 bit** per canale RGB (potendo rappresentare quindi valori che vanno da 0 a 255), avremo una **profondità di colore** di $8 * 3 = 24$ bit, per un totale di 2^{24} colori diversi.

È molto interessante imparare [come un'immagine viene catturata dal mondo reale e convertita in bit](#).

Un'altra proprietà delle immagini è la **risoluzione**, cioè il numero di pixel per ogni dimensione. Viene spesso indicata utilizzando la notazione "larghezza x altezza", ad esempio **4x4** per indicare un'immagine larga 4 pixel e alta 4 pixel.



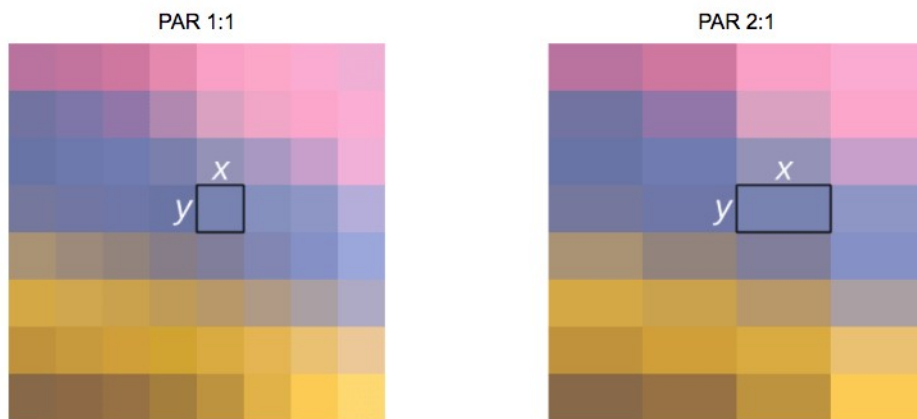
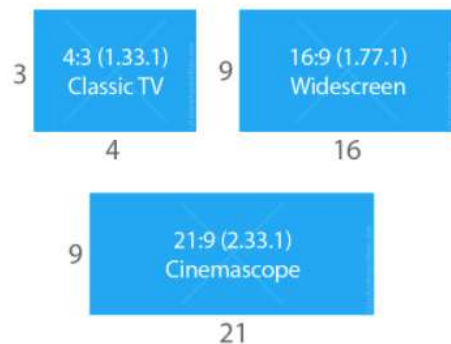
Esercizio: sperimentare con le immagini e i colori

Puoi esercitarti [con le immagini e i colori](#) utilizzando [jupyter](#) (python, numpy, matplotlib, ecc.).

Puoi anche imparare [come funzionano i filtri](#) (rilevazione dei contorni, nitidezza, sfocatura, ecc.).

Lavorando con immagini e video incontreremo anche una proprietà che si chiama **rapporto d'aspetto**, che descrive la relazione di proporzionalità tra la larghezza e l'altezza di un'immagine o di un pixel.

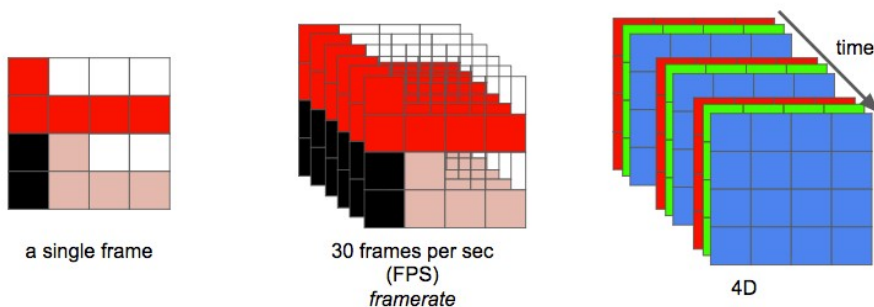
Quando senti dire che un film o un'immagine è in 16:9, di solito ci si riferisce al **rapporto d'aspetto dell'immagine** (anche chiamato *DAR*, dall'inglese *Display Aspect Ratio*). Tuttavia, anche i pixel possono avere forme diverse, per cui si parla di rapporto d'aspetto anche per i pixel (più precisamente *PAR*, dall'inglese *Pixel Aspect Ratio*).



I DVD hanno un DAR 4:3

Anche se la risoluzione video di un DVD è 704x480, il rapporto d'aspetto è 4:3, perché i pixel hanno un PAR di 10:11 (704x10/480x11).

Infine, possiamo definire un **video** come una **successione di n fotogrammi per unità di tempo**. Il valore n è la frequenza dei fotogrammi (*frame rate* in inglese), spesso abbreviata con FPS (fotogrammi al secondo).



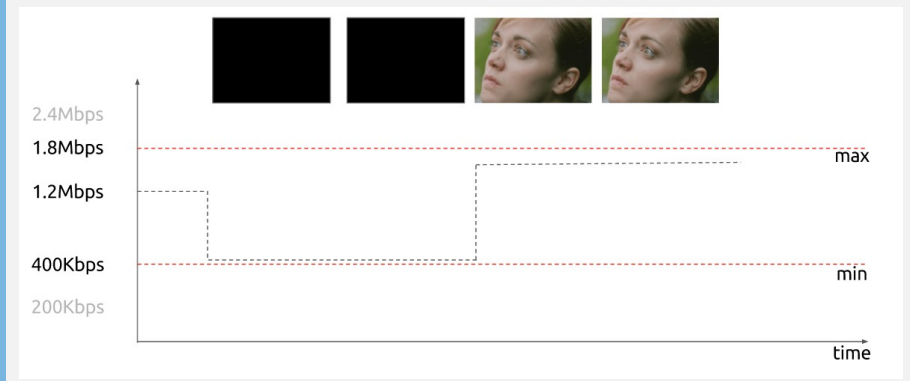
Il numero dei bit necessari per rappresentare un secondo di video è chiamato **bitrate**, o meno comunemente *frequenza di bit*.

$$\text{bitrate} = \text{larghezza} \times \text{altezza} \times \text{profondità di bit} \times \text{fotogrammi al secondo}$$

Ad esempio, un video con 30 fotogrammi al secondo e 24 bit per pixel, con una risoluzione di 480x240 ha bisogno (se non utilizziamo nessun tipo di compressione) di **82'944'000 bit al secondo**, cioè 82,944 Mbps (il calcolo è $30 \times 480 \times 240 \times 24$ byte).

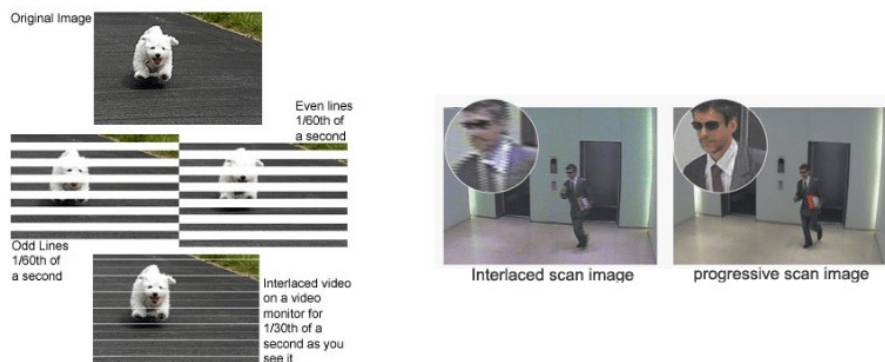
Quando il **bitrate** rimane quasi uguale nel tempo, parliamo di **bitrate costante (CBR)**, mentre quando varia di **bitrate variabile (VBR)**.

Il grafico mostra il bitrate di un video codificato in modalità VBR limitata (dai valori min e max). Si può notare come il bitrate è più basso quando il fotogramma è nero.



Molto tempo fa è stata inventata una tecnica per **raddoppiare il framerate percepito** da chi sta guardando un video, **senza richiedere l'utilizzo di dati aggiuntivi**. Questa tecnica è conosciuta come **interlacciament**. Semplificando, in un video interlacciato metà dell'immagine è contenuta in un fotogramma, mentre l'altra metà nel fotogramma successivo.

Al giorno d'oggi la maggior parte degli schermi utilizza la **scansione progressiva**, una tecnica per cui la trasmissione, memorizzazione e visualizzazione delle immagini in movimento avviene disegnando progressivamente e in sequenza tutte le linee che compongono il fotogramma.



A questo punto abbiamo un'idea di come un'immagine viene rappresentata digitalmente, come i suoi colori sono disposti, quanti **bit al secondo** sono richiesti per mostrare un video, con bitrate costante (CBR) o variabile (VBR), e cosa sono i concetti di **risoluzione**, **framerate** (FPS), interlacciamento, PAR, ecc.

Esercizio: controllare le proprietà di un file video

Puoi vedere la maggior parte delle proprietà che abbiamo appena visto [usando ffmpeg oppure mediainfo](#) su un file video.

Rimozione delle informazioni ridondanti

Abbiamo imparato che memorizzare un video senza nessuna compressione è impraticabile. **Un'ora di video** a una risoluzione 720p30 (720 linee verticali con scansione progressiva, con 30 fotogrammi al secondo)

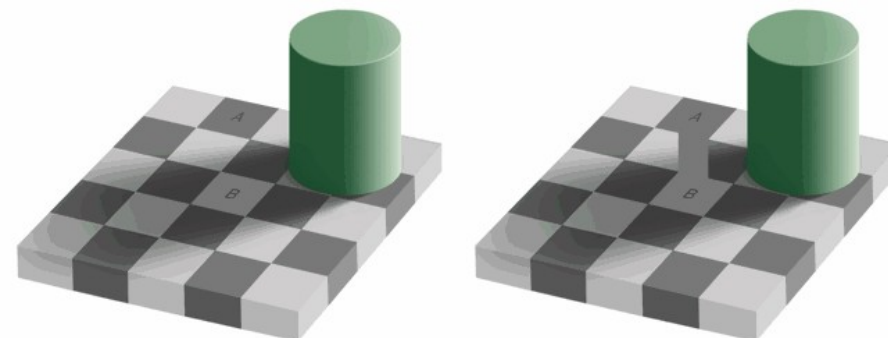
richiederebbe 278 G *. Dato che utilizzando i normali algoritmi di compressione lossless utilizzato in PKZIP, Gzip e PNG) non otterremmo una diminuzione significativa della dimensione del file, dobbiamo trovare altri modi per comprimere il video.

*Si può ottenere questo risultato moltiplicando $1280 \times 720 \times 24 \times 30 \times 3600$ (larghezza, altezza, bit per pixel, fps e durata in secondi).

Per farlo possiamo sfruttare il modo in cui la nostra vista funziona. Possiamo infatti sfruttare il fatto che l'occhio riesce a distinguere meglio diverse luminosità che diversi colori. È importante anche considerare le **ripetizioni nel tempo** (un video contiene molti fotogrammi molto simili, con poche differenze) e le **ripetizioni all'interno delle immagini** (un fotogramma può contenere aree che hanno lo stesso colore o un colore molto simile).

I colori, la luminanza e i nostri occhi

I nostri occhi sono **più sensibili alla luminosità che ai colori**, puoi vederlo tu stesso con l'immagine seguente.



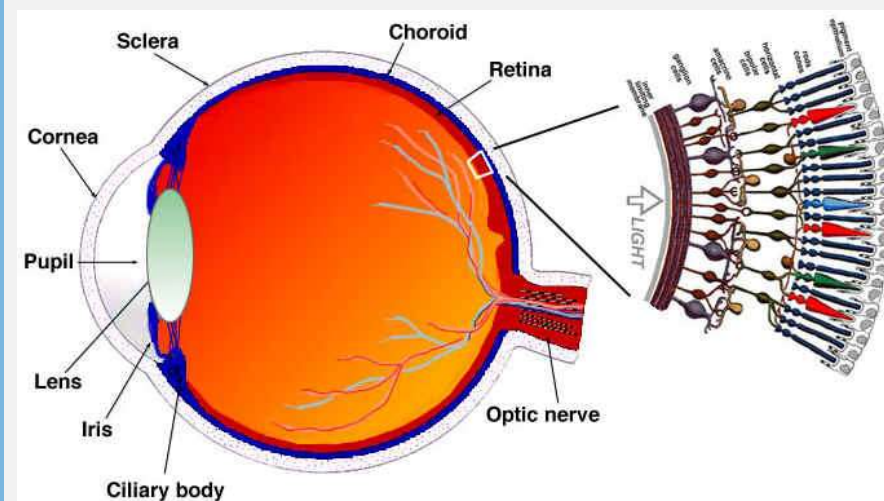
Se nell'immagine a sinistra non riesci a vedere che i colori dei quadrati A e B sono identici, è normale. È il nostro cervello che ci induce a **prestare più attenzione alla luce e al buio che al colore**. Nell'immagine a destra i due quadrati sono collegati con una striscia dello stesso colore dei quadrati, così riusciamo molto più facilmente a vedere che A e B sono effettivamente dello stesso colore.

Spiegazione semplificata del funzionamento dell'occhio

L'occhio è un **organo complesso**: è composto da molte parti ma siamo prevalentemente interessati alle cellule fotorecetttrici, i coni e i bastoncelli. L'occhio contiene **circa 120 milioni di bastoncelli e 6 milioni di coni**.

In modo **molto sempl**, proviamo ad associare colori e luminosità alle parti degli occhi. I **bastoncelli** sono quasi solo responsabili della luminosità, mentre i **coni** sono responsabili del colore. I coni sono di tre tipi, ognuno con un pigmento diverso e sensibile a un colore diverso. In particolare, esistono **coni S (blu)**, **coni M (verde)** e **coni L (rosso)**.

Dato che l'occhio ha molti più bastoncelli (luminosità) che coni (colore), si può dedurre che l'occhio è più bravo a distinguere il chiaro dallo scuro che i colori.



Curve di sensibilità al contrast

I ricercatori di psicologia sperimentale (e altri campi) hanno sviluppato molte teorie sull'occhio umano. Una di queste si chiama "curve di sensibilità al contrasto", e studia le funzioni che descrivono il legame spaziotemporale della luce, il loro valore data una certa quantità di luce iniziale, e quanto cambiamento è richiesto prima che un osservatore si accorga che c'è stato un cambiamento. Il plurale della parola "curve" è dovuto al fatto che possiamo usare queste funzioni non solo per studiare il bianco e il nero, ma anche i colori. Il risultato di questi esperimenti mostra che nella maggior parte dei casi l'occhio è più sensibile alla luce che al colore.

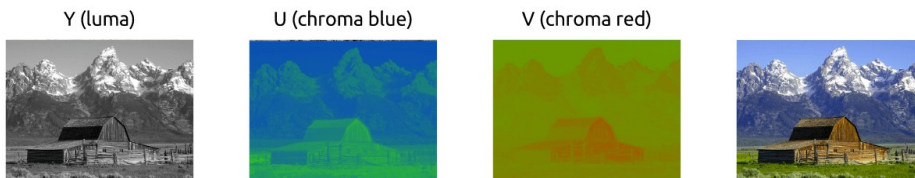
Ora che sappiamo che siamo più sensibili alla luminanza (la luminosità di un'immagine), possiamo provare a sfruttare questo fatto.

Modelli di colore

Abbiamo inizialmente imparato [come colorare le immagini](#) usando il **modello RGB**, ma esistono altri modelli. Ad esempio esiste un modello che separa la luminanza (la luminosità) dalla crominanza (i colori), ed è conosciuto come **YCb** *.

* esistono anche altri modelli che fanno la stessa distinzione.

Questo modello di colore utilizza la **Y** per rappresentare la luminosità, e due canali **Cb** (*chroma blue*) e **Cr** (*chroma red*) per i colori. Il modello **YCb** può essere derivato da RGB, e viceversa. Con i tre canali possiamo creare immagini pienamente colorate, come mostrato nell'immagine:



Conversione tra YCbCr e RGB

Qualcuno si potrebbe chiedere: **"come è possibile produrre tutti i colori senza usare il verde?"**

Per rispondere a questa domanda, proveremo a fare una conversione da RGB a YCbCr. Utilizzeremo coefficienti definiti dallo **standard BT.601**, una delle raccomandazioni del gruppo **ITU-R**.*

Il primo passo è **calcolare la luminanza**, utilizzando le costanti suggerite dall'ITU e i valori RGB.

$$Y = 0.299R + 0.587G + 0.114B$$

Ora che abbiamo la luminanza, possiamo **dividere i** (colore blu e rosso):

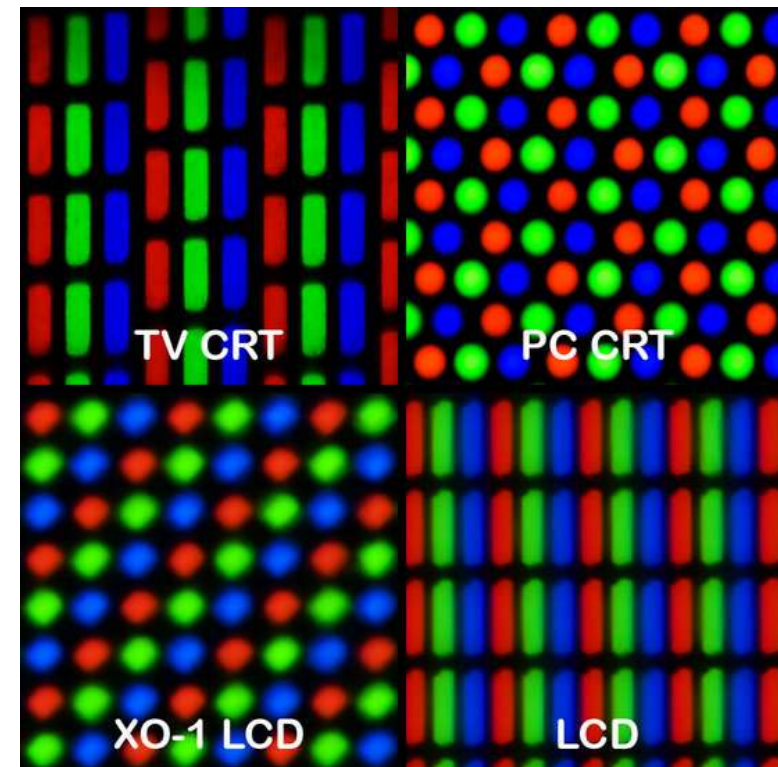
$$\begin{aligned} Cb &= 0.564(B - Y) \\ Cr &= 0.713(R - Y) \end{aligned}$$

Possiamo a **tornare indietro** facendo la conversione inversa, e anche **ottenere il valore del verde utilizzando YCbCr**.

$$\begin{aligned} R &= Y + 1.402Cr \\ B &= Y + 1.772Cb \\ G &= Y - 0.344Cb - 0.714Cr \end{aligned}$$

* i gruppi e gli standard sono comuni nel mondo del video digitale, di solito definiscono quali sono gli standard da usare, ad esempio **cosa si intende con 4K?** **quale framerate dovrei usare?** **quale risoluzione e modello di colore?**

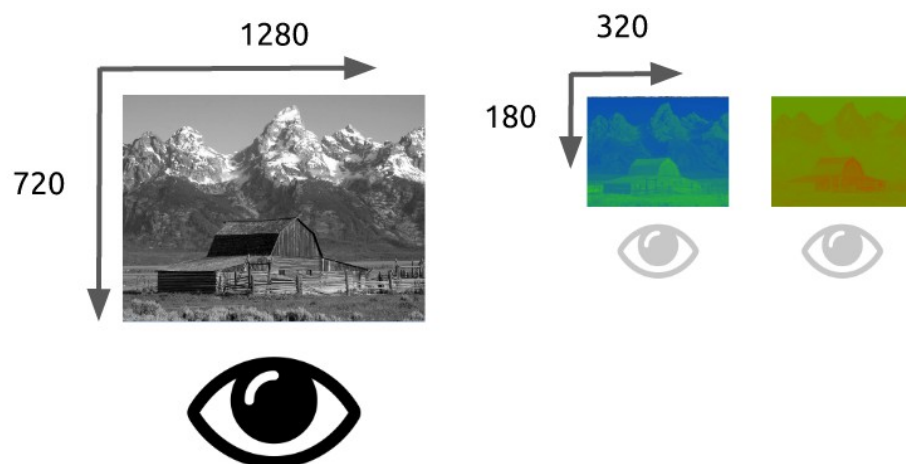
In genere i display (monitor, tv, schermi, ecc.) utilizzano **solo il modello RGB**, anche se con geometrie dei pixel diverse, come si vede nelle immagini, che rappresentano schermi di tipo diverso, ingranditi.



Sottocampionamento della crominanza

Con la rappresentazione dell'immagine in termini di luminanza e crominanza, possiamo trarre vantaggio dalle caratteristiche dell'occhio (che privilegiano la luminanza) per rimuovere selettivamente delle informazioni. La

tecnica di **sottocampionamento della cromaticità** fa proprio questo, e cioè utilizza **meno risoluzione per la cromaticità rispetto alla luminanza**.



Quanto dovremmo ridurre la risoluzione della cromaticità? Si scopre che esistono già degli schemi che descrivono come gestire la risoluzione e l'unione **colore finale** = $Y + Cb + Cr$.

Questi schemi sono noti come **sistemi di sottocampionamento** e sono espressi utilizzando un rapporto composto da tre valori. In particolare **a:x:y** definisce la risoluzione di cromaticità di un blocco di pixel di luminanza di dimensione **a x 2** pixel.

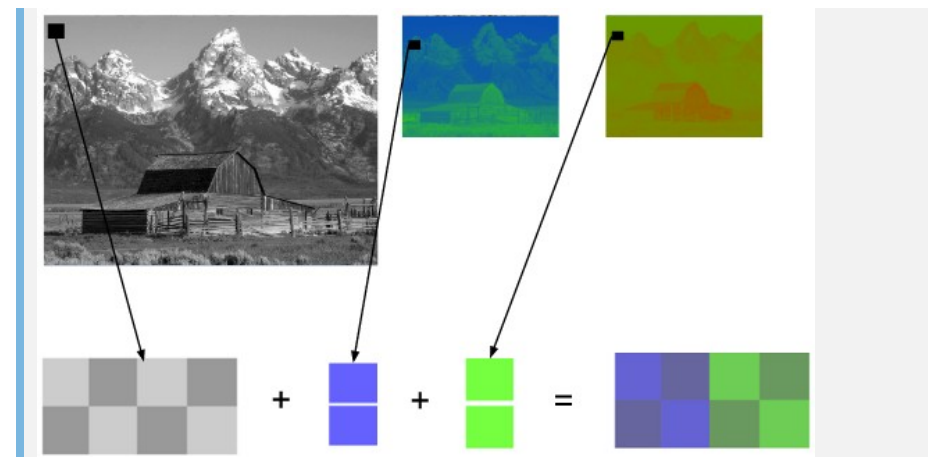
- **a** numero di pixel orizzontali di riferimento per il campionamento (di solito 4)
- **x** numero di campioni di cromaticità nella prima riga di **a** pixel (risoluzione orizzontale in relazione a **a**)
- **y** numero di cambiamenti di campioni di cromaticità tra la prima e la seconda riga della griglia **a x 2**

C'è un'eccezione a questa definizione, in particolare nello schema 4:1:0, che prevede un singolo campione di cromaticità per ogni blocco **4 x 4** di luminanza.

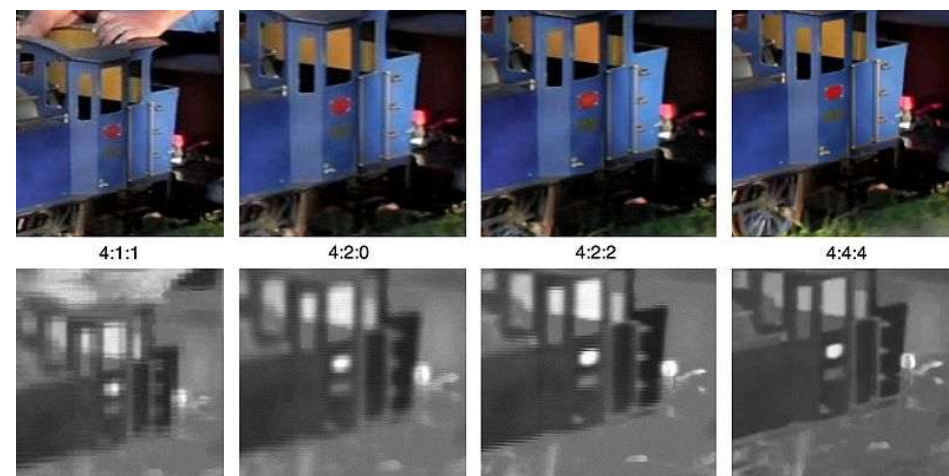
Tra gli schemi più comuni nei codec moderni ci sono **4:4:4** (nessun sottocampionamento), **4:2:2**, **4:1:1**, **4:2:0**, **4:1:0** and **3:1:1**.

Unione di YCbCr 4:2:0

Ecco un esempio di come funziona l'unione di un'immagine usando YCbCr con schema di sottocampionamento della cromaticità 4:2:0. Nota che vengono utilizzati soltanto 12 bit per pixel.



Le immagini che seguono mostrano la stessa foto codificata con diversi tipi di sottocampionamento della cromaticità. Le immagini della prima riga sono quelle finali, mentre quelle della seconda riga mostrano la risoluzione della cromaticità. Il risultato è ottimo dato che la perdita di qualità nell'immagine a colori è molto bassa.



Prima abbiamo calcolato che sono necessari **278 GB di spazio per archiviare un file video dalla durata di un'ora, con risoluzione 720p e framerate di 30 fps**. Se utilizziamo il modello colore **YCbCr 4:2:0**, possiamo **dimezzare la dimensione (139 GB)***, anche se siamo ben lontani dall'ideale.

*si ottiene questo valore moltiplicando tra loro larghezza, altezza, bit per pixel e fps. Con il modello RGB avevamo bisogno di 24 bit per pixel, ora ne abbiamo bisogno soltanto

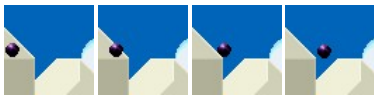
Esercizio: vedere l'istogramma YCbCr

Puoi controllare l'istogramma di un video YCbCr con [ffmpeg](#). Ad esempio, in questa scena il colore blu porta un alto contributo, come evidenziato dall'istogramma.



Tipi di fotogramma

Prima di occuparci della **ridondanza delle informazioni nel tempo**, chiariamo la terminologia che useremo. Supponiamo di avere un filmato con 30 fps, i cui primi 4 fotogrammi sono:



È evidente che ci sono **molte ripetizioni** all'interno e tra i fotogrammi. Ad esempio lo sfondo blu non cambia tra il primo e l'ultimo fotogramma. Per sfruttare questa caratteristica dei fotogrammi, possiamo **categorizzare in modo astratto** i fotogrammi in **tre tipi diversi**.

I-frame (intra, keyframe)

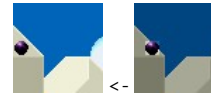
Un I-frame (chiamato anche *reference frame*, *keyframe*, *intra-frame*) è un fotogramma **indipendente**. Non si affida a nessun'altra informazione esterna per essere renderizzato. È come una foto statica. Di solito il primo fotogramma di un video è un I-frame, ma vedremo che ce ne possono essere più di uno in un video, mischiati tra gli altri fotogrammi di tipo diverso.



P-frame (predicted)

Un P-frame (*fotogramma predetto*) sfrutta il fatto che quasi sempre il fotogramma può essere **renderizzato utilizzando il fotogramma precedente**. Ad esempio, nel secondo fotogramma l'unico cambiamento rispetto

al precedente è il movimento della palla. Possiamo quindi **ricostruire questo fotogramma utilizzando le differenze rispetto al precedente**.



Esercizio: un video con un solo I-frame

Dato che un P-frame utilizza meno dati rispetto a un I-frame, perché non possiamo codificare un intero video **usando solo un I-frame e poi solo P-frame**?

Per esercizio codifica il video esempio, riproducilo **salta a una posizione avanzata** del video. Noterai che sarà **richiesto del tempo** perché il player completi l'operazione di *seek*. Questo avviene perché un **P-frame ha bisogno di un fotogramma di riferimento** (ad esempio un I-Frame, ma non per forza) per essere renderizzato.

Un altro veloce esperimento che puoi fare è **codificare un video sia utilizzando un singolo I-frame che impostato un I-frame ogni 2 secondi**, e confrontare la **dimensione dei due file risultanti**.

B-frame (bi-predictive)

E se potessimo fare riferimento sia a fotogrammi passati che futuri, per ottenere una compressione migliore? È proprio come funzionano i B-frame, o frame bi-predittivi.

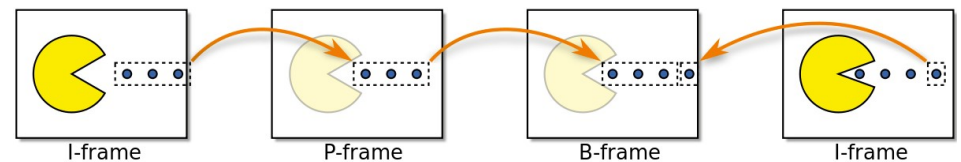


Esercizio: confronto video con e senza B-frame

Genera due video, uno con i B-frame e uno **senza B-frame**, e confronta la differenza di dimensione del file e la qualità dei risultati.

Sommario

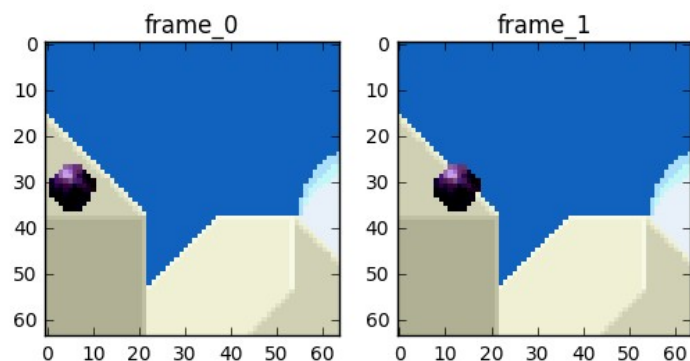
Diversi tipi di frame possono essere utilizzati per **fornire una migliore compressione**. Scopriremo meglio come nella prossima sezione, ma per ora è importante capire che **gli I-frame sono costosi, i P-frame un po' meno, e infine i B-frame sono i più leggeri**.



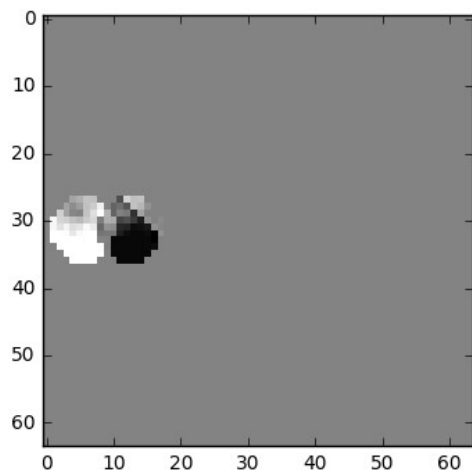
Ridondanza temporale (predizione inter-frame)

Esploriamo le possibilità che abbiamo per cercare di ridurre la **ripetizione delle informazioni nel tempo**. Questo tipo di ridondanza può essere affrontata usando la **predizione tra fotogrammi** (*inter-frame prediction* o *inter prediction*).

L'obiettivo è **utilizzare meno bit** per codificare la sequenza di fotogrammi 0 e 1.



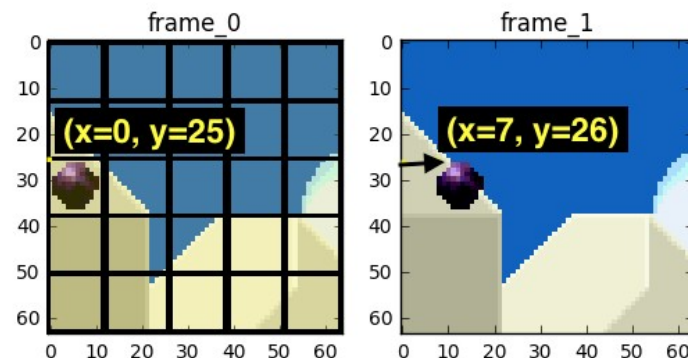
Una cosa che potremmo fare è **sottrarre il fotogramma 1 dal fotogramma 0**, in modo da ottenere soltanto il **residuo da codificare**.



E se ti dicessi che esiste un **metodo miglior**, che consente di utilizzare ancora meno informazione? Consideriamo il **fotogramma 0** come un insieme di partizioni (riquadri) ben definite, e proviamo a creare un'associazione tra i blocchi del **fotogramma 0** e il **fotogramma 1**. Possiamo pensare a questo meccanismo come una **stima del moto**.

Da Wikipedia: compensazione del moto dei blocchi

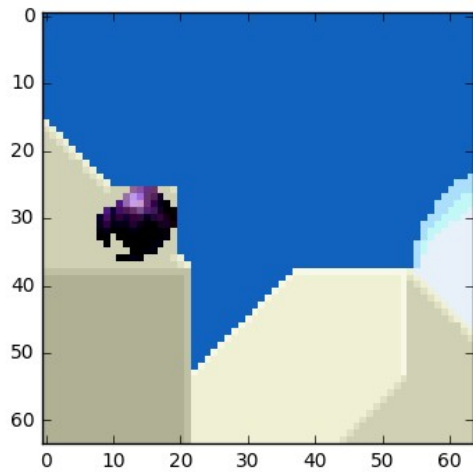
"La **compensazione del moto dei blocchi** divide il fotogramma corrente in blocchi non sovrapposti, e il vettore di compensazione del moto **indica da dove si sono spostati quei blocchi** (una convinzione comune ma errata è che il fotogramma precedente viene diviso in blocchi non sovrapposti, e i vettori di compensazione del moto indicano dove i blocchi si sposteranno). I blocchi sorgente tipicamente si sovrappongono nel fotogramma sorgente. Alcuni algoritmi di compressione video costruiscono il fotogramma corrente utilizzando pezzi di diversi fotogrammi creati precedentemente."



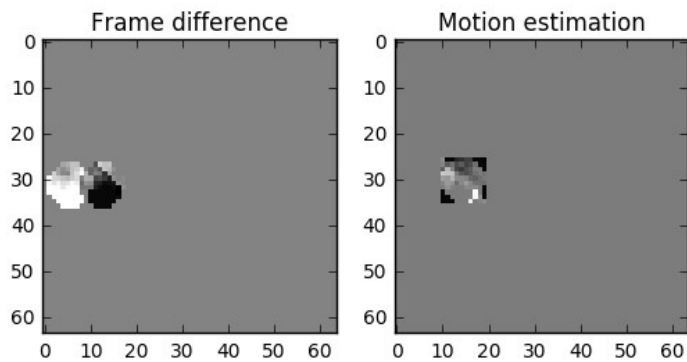
Possiamo stimare che la palla si è spostata dalla posizione $x=0$, $y=25$ alla posizione $x=6$, $y=26$. I valori di x e y determinano i **vettori del moto**. Un passo aggiuntivo che possiamo fare per risparmiare spazio è **codificare soltanto il vettore di spostamento ottenuto dalla differenza** tra l'ultima posizione del blocco e quella predetta. Nell'esempio sopra il vettore del moto sarebbe quindi $x=6$ ($6-0$), $y=1$ ($26-25$).

In una situazione di codifica reale, **la palla sarebbe divisa tra più partizioni**, ma il procedimento è lo stesso.

Gli oggetti nel fotogramma **si muovono in 3 dimensioni**, cioè ad esempio la palla può diventare più piccola se si muove verso lo sfondo. È quindi normale non trovare **la corrispondenza perfetta** tra blocchi. Ad esempio questo è il risultato della nostra stima del moto in confronto alla figura reale:

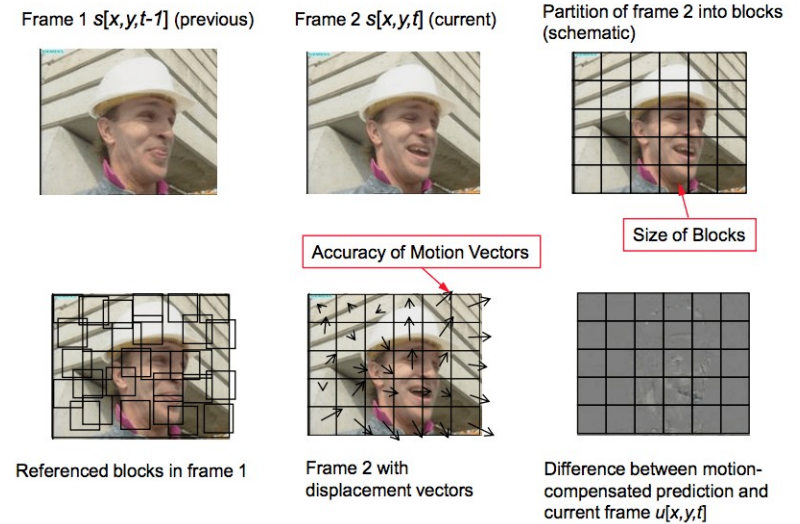


Il vantaggio di applicare la tecnica della **stima del moto** è che i **dati da codificare sono di meno** in confronto a una semplice differenza tra fotogrammi.



Come funziona realmente la compensazione del moto

La tecnica appena vista viene applicata a tutti i blocchi, ma probabilmente la palla dell'esempio verrebbe partizionata in più di un blocco.

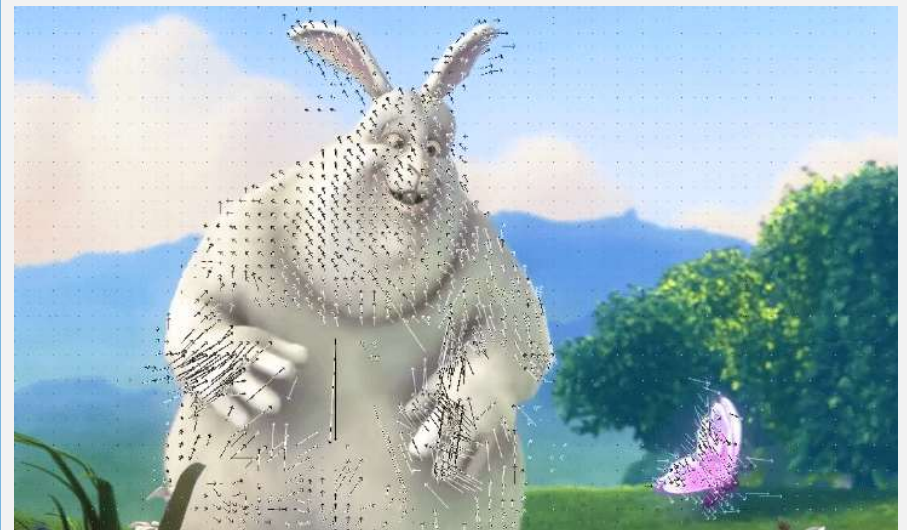


Fonte: https://web.stanford.edu/class/ee398a/handouts/lectures/EE398a_MotionEstimation_2012.pdf

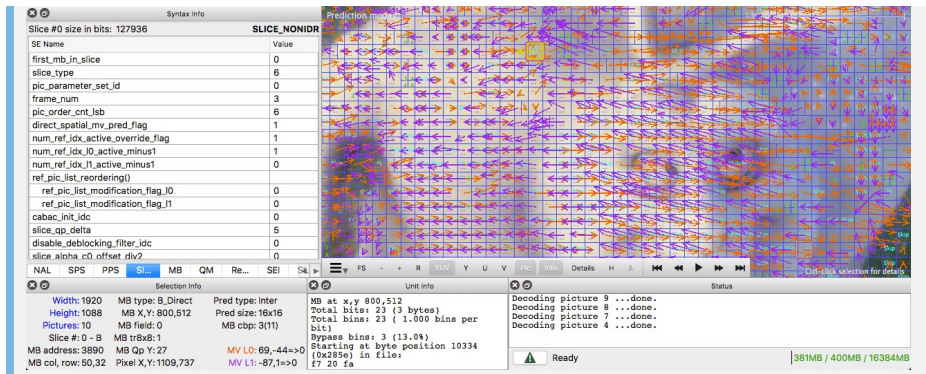
Puoi esercitarti con questi concetti [usando jupyter](#).

Esercizio: vedere i vettori del m

Possiamo generare un video [che mostra la predizione tra frame \(e i vettori del moto\)](#) con `ffmpeg`.

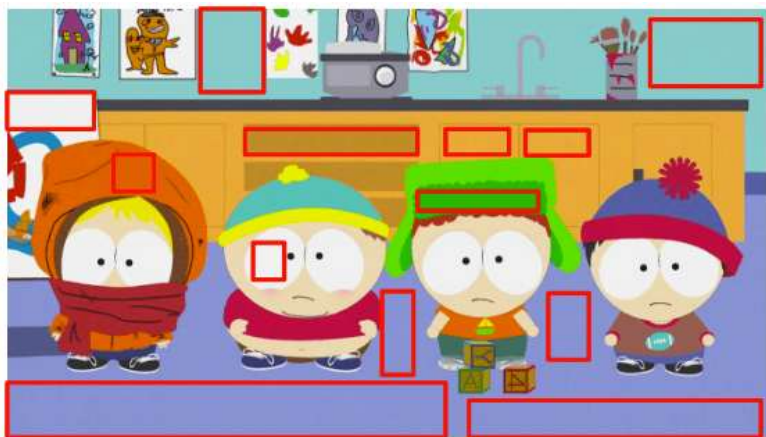


In alternativa possiamo usare il software [Intel Video Pro Analyzer](#) (è a pagamento ma esiste una versione di prova che limita l'analisi ai primi 10 fotogrammi).



Ridondanza spaziale (predizione intra-frame)

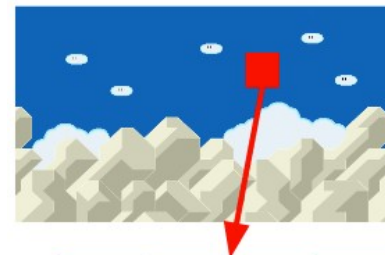
Se analizziamo un **singolo fotogramma** di un video noteremo che ci sono **molte aree correlate**.



Facciamo un esempio. Questa scena è composta prevalentemente dai colori blu e bianco:



Questo fotogramma è un **I-frame**, per cui **non possiamo usare fotogrammi precedenti** per costruirlo. Possiamo però comunque comprimerlo. Proviamo ad esempio a codificare il blocco evidenziato in rosso: guardiamo **intorno al blocco** possiamo **stima** che c'è una **tendenza di colore attorno al b**.



100	100	100	200
100	???	???	???
100	???	???	???
100	???	???	???

unknown values

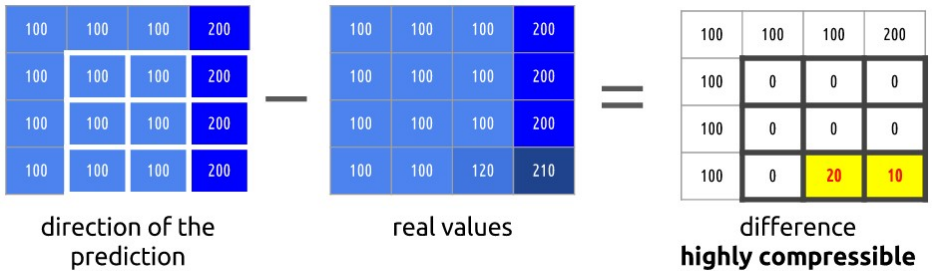
Possiamo ad esempio **predire** che i pixel colorati sopra il blocco **continuo allo stesso modo verticalmente**. In altre parole **i pixel del blocco che hanno colore sconosciuto assumeranno attraverso la predizione il colore dei pixel che si trovano sopra** (in questo caso).



100	100	100	200
100	100	100	200
100	100	100	200
100	100	100	200

direction of the prediction

La **predizione che abbiamo fatto può essere sbagliata**, per cui dopo aver applicato la predizione dobbiamo **sottrarre i valori reali** dei pixel, in modo da ottenere un blocco residuo. Il risultato è una matrice molto più facilmente comprimibile in confronto all'originale.

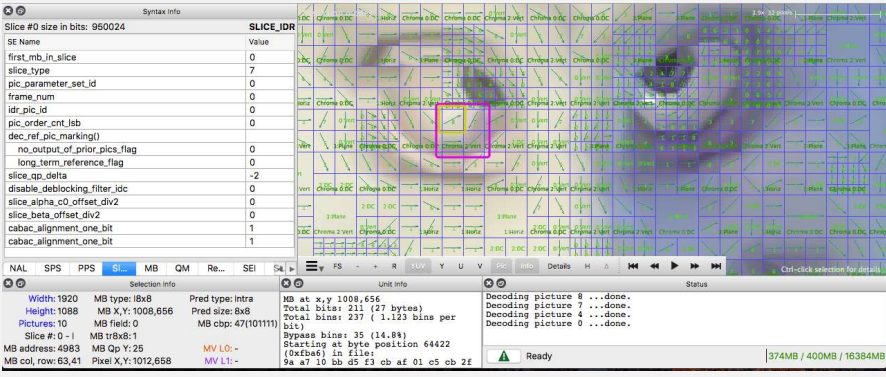


Esercizio: vedere la predizione intra-frame

Puoi [generare un video con i macro-blocchi e le sue predizioni con ffmpeg](#). Consulta la documentazione di ffmpeg per comprendere meglio [il significato dei colori dei blocchi](#).



In alternativa puoi anche usare il software [Intel Video Pro Analyzer](#) (è a pagamento ma esiste una versione di prova che limita l'analisi ai primi 10 fotogrammi).



Come funziona un codec video?

Cosa? Perché? Come?

Cos'è? È un software o un dispositivo hardware che comprime o decomprime un video digitale. **Perché?** Il mercato e la società richiedono video di qualità alta con banda o spazio di archiviazione limitati. Ti ricordi che abbiamo [calcolato la banda necessaria](#) per trasmettere un video con 30 fotogrammi al secondo, 24 bit per pixel e una risoluzione di 480x240 pixel? Risultava **83 Mbps**, senza applicare alcuna compressione. I codec sono l'unico modo per fornire video HD/Full HD/UHD alle tv e tramite Internet. **Come?** Stiamo per dare un'occhiata alle principali tecniche usate.

CODEC vs Container

Un errore che viene spesso fatto dai principianti è confondere il CODEC video con il [contenitore video](#). Possiamo pensare al **contenitor** come un formato "wrapper" che contiene i metadati sulle tracce video (e audio), e il vero e proprio **video compresso** come carico pagante (*payload*).

Solitamente l'estensione di un file video determina il suo formato contenitore. Ad esempio, un file [video.mp4](#) indica con molta probabilità il formato **MPEG-4 Part 14**, mentre un file chiamato [video.mkv](#) si riferisce al formato **Matroska**. Per controllare con certezza il codec e il formato contenitore di un file possiamo utilizzare [ffmpeg oppure mediainfo](#).

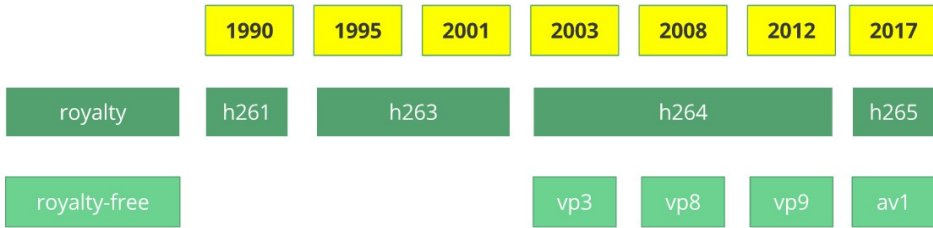
Storia

Prima di passare ad analizzare il funzionamento interno di un codec generico, facciamo un salto indietro nella storia per comprendere meglio alcuni vecchi codec.

Il codec video [H.261](#) nacque nel 1990 (tecnicamente nel 1988) e fu progettato per funzionare con **bitrate di 64 kbit/s**. Il codec utilizzava già idee come il sottocampionamento della crominanza, i macro-blocchi, ecc. Nell'anno 1955 lo standard del codec video [H.263](#) fu pubblicato, poi modificato ed esteso fino al 2001.

Nel 2003 la prima versione del codec **H.264/AVC** fu completata. Nello stesso anno, un'azienda chiamata **TrueMotion** rilasciò un altro codec per la compressione video, lossy e **royalty-free**, chiamato **VP3**. Nel 2008, **Google acquistò l'azienda**, rilasciando **VP8** nello stesso anno. Nel dicembre del 2012, Google rilasciò **VP9**, oggi **supportato da circa ¾ dei browser sul merca** (mobile incluso).

AV1 è un nuovo codec video **royalty-free** e open source che è stato progettato dalla [Alliance for Open Media \(AOMedia\)](#), composta da grandi aziende come **Google, Mozilla, Microsoft, Amazon, Netflix, AMD, ARM, NVidia, Intel e Cisco**. La **prima versione 0.1.0** del codec di riferimento è stata pubblicata il 7 aprile 2016.



La nascita di AV1

All'inizio del 2015, Google stava lavorando su [VP10](#), Xiph (Mozilla) su [Daala](#) e Cisco ha reso open source il suo codec video royalty-free chiamato [Thor](#).

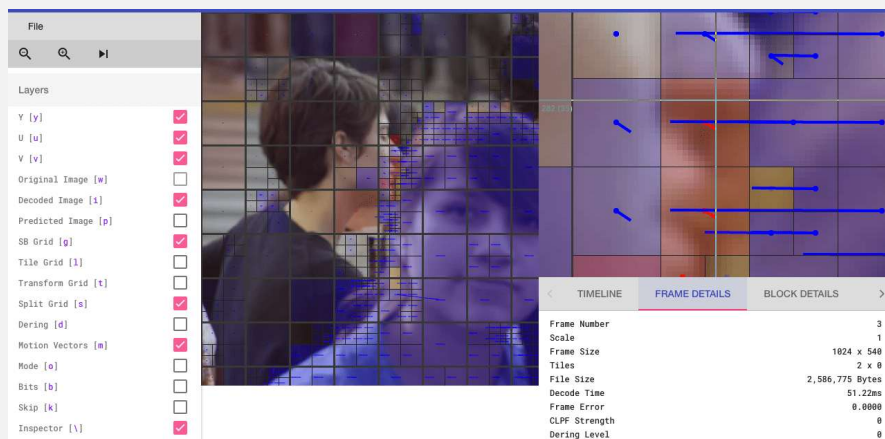
MPEG LA ha annunciato dei limiti annuali per HEVC (H.265) e canoni di utilizzo 8 volte superiori a H.264, per poi cambiare le regole di nuovo:

- **nessun limite annuale,**
- **commissione sui contenuti** (lo 0,5% delle entrate)
- **commissioni per unità circa 10 volte più alte di H.264.**

La [Alliance for Open Media](#) è stata creata da aziende produttrici di hardware (Intel, AMD, ARM, Nvidia, Cisco), aziende che forniscono contenuti (Google, Netflix, Amazon), browser (Google, Mozilla), e altre.

Le aziende si sono poste un obiettivo comune: un video codec royalty-free. È quindi nato AV1, con un [sistema di licenze molto più semplice](#). [Timothy B. Terriberry](#) ha fatto [un'ottima presentazione](#) (fonte di questa sezione) sul concepimento di AV1, il modello di licenza e lo stato attuale del codec.

Ti sorprenderà sapere che puoi **analizzare il codec AV1 tramite il tuo browser**, andando alla pagina <https://arewecompressedyet.com/analyzer/>



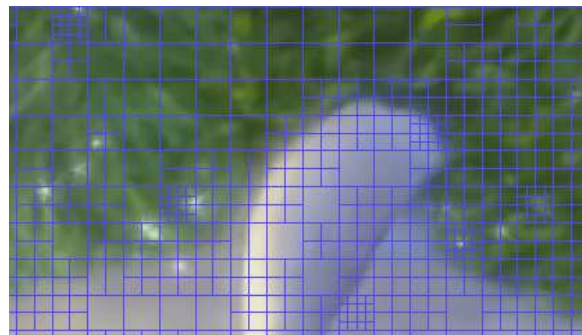
P.S.: se vuoi imparare di più sulla storia dei codec è importante conoscere le basi che stanno dietro ai [brevetti legati alla compressione video](#).

Un codec generico

Introdurremo ora i **meccanismi principali che stanno dietro a un generico codec video**, ma che si applicano anche ai principali codec moderni come VP9, AV1 e HEVC. Tieni presente che la spiegazione sarà *molto* semplificata rispetto alla realtà, anche se useremo qualche esempio reale (come H.264) per mostrare il funzionamento di una tecnica.

1° passo - partizionamento dell'immagine

Il primo passo è **dividere il fotogramma** in molte **partizioni, sotto-partizioni** e via dicendo.



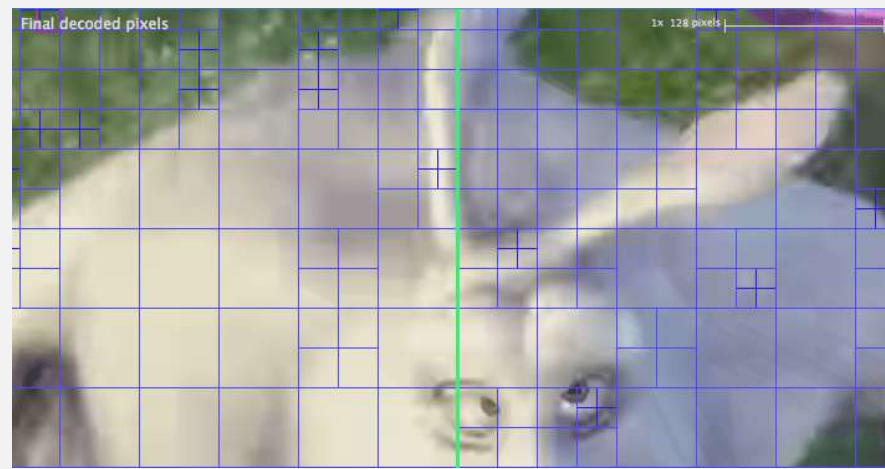
Perché si fa Ci sono molte ragioni: ad esempio, dividendo l'immagine possiamo calcolare le predizioni più precisamente, e utilizzare le partizioni più piccole per le parti in movimento, mentre quelle più grandi per rappresentare sfondi statici.

Solitamente, i CODEC **organizzano le partizioni** in *slice* (o *tile*), *macro* (o *coding tree unit*) e tante altre sotto-partizioni. La dimensione massima di queste partizioni varia, ad esempio per HEVC è 64x64 pixel, per AVC (H.264) è 16x16, con le sotto-partizioni che possono essere piccole fino a 4x4 pixel.

Ti ricordi che i fotogrammi possono essere di **diverso tipo**? Si può **applicare la stessa idea anche ai blocchi**, per cui possiamo avere I-slice, B-slice, I-macroblock, ecc.

Esercizio: vedere le partizioni

Puoi usare il software [Intel Video Pro Analyzer](#) (è a pagamento ma esiste una versione di prova che limita l'analisi ai primi 10 fotogrammi). Ecco un esempio che mostra le [partizioni di VP9](#).



2° passo - predizioni

Ora che abbiamo le partizioni, possiamo usarle per fare le predizioni. Per la predizione **inter-frame** dobbiamo raccogliere informazioni sui **vettori del moto e sul residuo**, mentre per la predizione **intra-frame** ci servono **la direzione della predizione e il residuo**.

3° passo - trasformazione

Una volta che abbiamo il blocco residuo (ottenuto da **partizione predetta - partizione reale**), possiamo **trasformarlo** in un modo che ci consenta di vedere **quali pixel possiamo scartare** pur mantenendo la **qualità generale**. Ci sono diverse trasformazioni che ci permettono di farlo.

Anche se esistono **molte altre trasformazioni**, daremo uno sguardo più approfondito alla trasformata discreta del coseno (**DCT**). La **DCT** ci permette di:

- **convertire** blocchi di pixel in blocchi di **coefficienti di frequenza** della stessa dimensione
- **compattare** l'energia, aiutandoci a eliminare la ridondanza spaziale
- **invertire** il processo, cioè tornare a blocchi di pixel

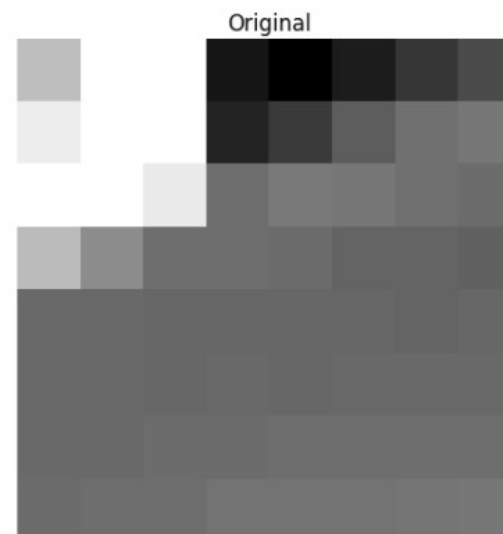
Il 2 febbraio 2017, Cintra, R. J. e Bayer, F. M hanno pubblicato l'articolo scientifico [DCT-like Transform for Image Compression Requires 14 Additions Only](#).

Non preoccuparti se non hai capito a pieno i benefici di ciascun punto, faremo qualche esperimento in modo da capire meglio il reale valore della trasformata.

Prendiamo il seguente **blocco di pixel** (8x8):

```
[ [227, 255, 255, 155, 146, 158, 169, 178],
  [247, 255, 255, 161, 171, 186, 194, 197],
  [255, 255, 246, 193, 198, 197, 194, 192],
  [226, 206, 193, 193, 192, 189, 189, 188],
  [191, 191, 190, 190, 190, 190, 189, 190],
  [191, 191, 190, 191, 190, 191, 191, 191],
  [191, 191, 192, 192, 193, 193, 193, 193],
  [192, 193, 193, 195, 195, 195, 196, 197] ]
```

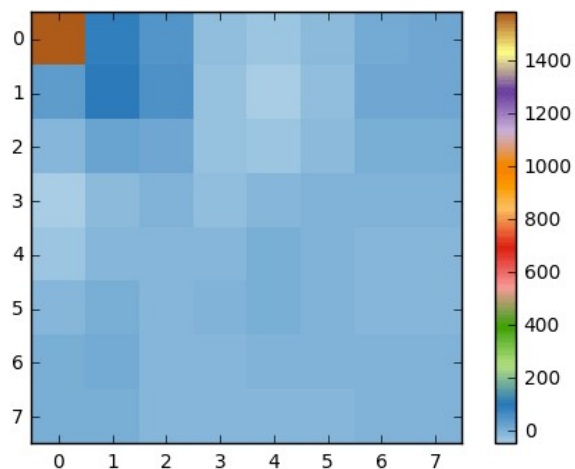
Che corrisponde alla seguente immagine (8x8):



Applicando la DCT a questo blocco di pixel otteniamo un **blocco dei coefficienti** (8x8):

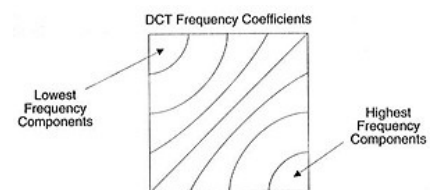
```
[ [ 1582. ,    87.2,    51.8,   -15.9,   -34.2,   -12.8,    14. ,    17.4 ],
  [   38.5,    96.1,    56. ,   -24.7,   -44.4,   -15.5,    17.1,    20.9 ],
  [   -3.2,    24.8,    20.3,   -26.2,   -28.9,    -9. ,     8.6,     9.8 ],
  [  -47.1,   -9.9,    -1.5,   -15.9,    -8.3,    -1.2,     0.9,    -1.4 ],
  [  -33. ,   -8.3,    -5. ,    -3. ,     7.8,     3.5,    -4.2,    -6. ],
  [   -8.3,     8.5,   -3.5,     2.3,     7.9,     2.9,    -3.7,    -4. ],
  [    8.6,    12.1,   -5.1,   -3.3,    -0.5,   -1.5,    -2. ,   -1.8 ],
  [    7.4,     5.8,   -6.7,   -5.9,    -5. ,   -3.8,     0.4,     0.9 ] ]
```

Il blocco dei coefficienti ottenuto, convertito in un'immagine, appare così:



Come puoi vedere non assomiglia per niente all'immagine originale. Si può anche notare che il **primo coefficiente** è molto diverso dagli altri. Questo perché il primo coefficiente della DCT è quello che rappresenta **tutti i campioni** della matrice di ingresso, qualcosa di **simile a una media**.

Il blocco dei coefficienti ha un'interessante proprietà, e cioè che separa i componenti relativi alle alte frequenze da quelli delle basse frequenze.



In un'immagine, **la maggior parte dell'energia** è concentrata nelle **basse frequenze**, quindi dopo aver applicato la trasformata possiamo **scartare i coefficienti delle alte frequenze**, in modo da **ridurre la quantità di dati** richiesta per descrivere l'immagine, pur senza sacrificare troppo la qualità dell'immagine.

la frequenza determina quando rapidamente un segnale cambia.

Proviamo ad applicare le conoscenze acquisite: convertiamo l'immagine originale nelle sue frequenze usando la DCT, e poi scartiamo i coefficienti meno importanti.

Per prima cosa, convertiamo il blocco nel **dominio della frequenza**.

```
[ [ 1582. , 87.2, 51.8, -15.9, -34.2, -12.8, 14. , 17.4 ],
[ 38.5, 96.1, 56. , -24.7, -44.4, -15.5, 17.1, 20.9 ],
[ -3.2, 24.8, 20.3, -26.2, -28.9, -9. , 8.6, 9.8 ],
[ -47.1, -9.9, -1.5, -15.9, -8.3, -1.2, 0.9, -1.4 ],
[ -33. , -8.3, -5. , -3. , 7.8, 3.5, -4.2, -6. ],
[ -8.3, 8.5, -3.5, 2.3, 7.9, 2.9, -3.7, -4. ],
[ 8.6, 12.1, -5.1, -3.3, -0.5, -1.5, -2. , -1.8 ],
[ 7.4, 5.8, -6.7, -5.9, -5. , -3.8, 0.4, 0.9 ]]
```

Poi, scartiamo parte dei coefficienti (il 67%), prevalentemente nell'angolo in basso a destra del blocco.

```
[ [ 1582. , 87.2, 51.8, -15.9, -34.2, -12.8, 0. , 0. ],
[ 38.5, 96.1, 56. , -24.7, -44.4, 0. , 0. , 0. ],
[ -3.2, 24.8, 20.3, -26.2, 0. , 0. , 0. , 0. ],
[ -47.1, -9.9, -1.5, 0. , 0. , 0. , 0. , 0. ],
[ -33. , -8.3, 0. , 0. , 0. , 0. , 0. , 0. ],
[ -8.3, 0. , 0. , 0. , 0. , 0. , 0. , 0. ],
[ 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ],
[ 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]]
```

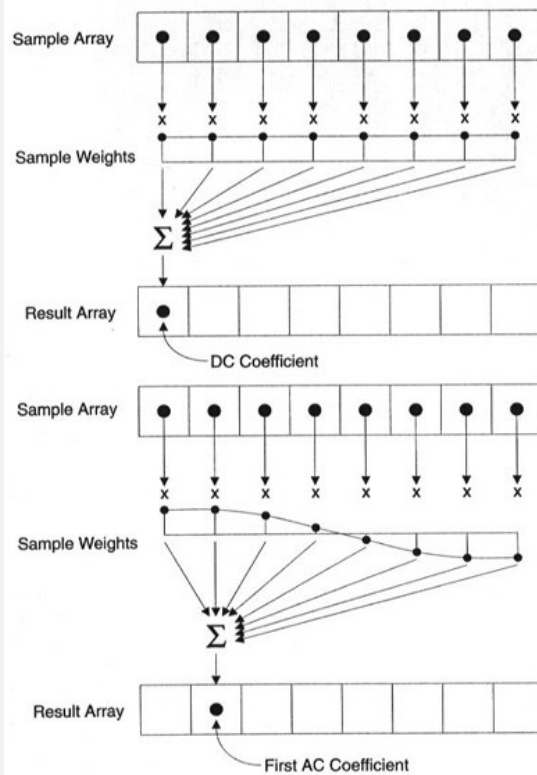
Infine, ricostruiamo l'immagine utilizzando il blocco dei coefficienti modificato, e confrontiamo il risultato con l'immagine originale.



L'immagine a destra è simile a quella originale, ma introduce anche molte differenze. Abbiamo però **buttato via il 67,1875%** delle informazioni e comunque siamo in grado di ottenere un'immagine che assomiglia a quella originale. Il prossimo passo è capire come modificare i coefficienti in modo più intelligente, per ottenere così una migliore qualità dell'immagine.

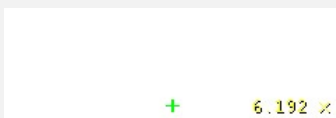
Ogni coefficiente viene calcolato utilizzando tutti i pixel

È importante evidenziare che ogni coefficiente del blocco non corrisponde direttamente a un singolo pixel, ma è una somma pesata di tutti i pixel. Il grafico che segue mostra come il primo e il secondo coefficiente vengono calcolati, utilizzando persi che sono diversi per ogni indice.



Fonte: <https://web.archive.org/web/20150129171151/https://www.iem.thm.de/telekom-labor/zinke/mk/mpeg2beg/whatisit.htm>

Puoi anche provare a [osservare visivamente la DCT con la formazione di un'immagine](#). Ad esempio, qua puoi vedere [la formazione della lettera A](#) utilizzando diversi pesi per i coefficienti.



Esercizio: scartare diversi coefficienti

Puoi sperimentare con [la trasformata DCT](#).

4° passo - quantizzazione

Quando scartiamo dei coefficienti, come abbiamo fatto alla fine dell'ultimo passo, stiamo già facendo una sorta di quantizzazione. Questo passo è quello in cui decidiamo quali informazioni "perdere", e lo facciamo **quantizzando i coefficienti per ottenere della compressione**.

Come possiamo quantizzare un blocco di coefficienti? Un metodo semplice potrebbe essere una quantizzazione uniforme, in cui prendiamo un blocco e **dividiamo tutti i valori per lo stesso numero** (10), per poi arrotondare.

120	40	1	0
45	3	0	0
-5	0	0	1
0	0	-2	0

 $\div \text{Qstep (10)} =$

12	4	0	0
4	0	0	0
0	0	0	0
0	0	0	0

Come possiamo **invertire** (ri-quantizzare) il blocco dei coefficienti? È sufficiente **moltiplicare i valori ottenuti per lo stesso numero** con cui li abbiamo divisi (10).

12	4	0	0
5	0	0	0
0	0	0	0
0	0	0	0

 $\times \text{Qstep (10)} =$

120	40	0	0
40	0	0	0
0	0	0	0
0	0	0	0

Questo approccio non è però il migliore, perché non tiene in considerazione l'importanza dei diversi coefficienti. Anziché un singolo valore per la divisione, potremmo utilizzare una **matrice di quantizzazione**, che sfrutterebbe quindi le proprietà della DCT quantizzando maggiormente l'angolo in basso a destra e in modo minore quello in alto a sinistra. La [compressione JPEG](#) utilizza un metodo simile, e puoi vedere un esempio di matrice di quantizzazione [nel codice sorgente](#) di un compressore JPEG.

Esercizio: quantizzazione

Puoi sperimentare [con la quantizzazione](#).

5° passo - codifica dell'entropia

Dopo aver quantizzato i dati (i blocchi/slice/fotogrammi), possiamo comprimerli ulteriormente utilizzando metodi di compressione lossless (senza perdita). Ci sono moltissimi algoritmi per comprimere i dati, e andremo a vederne alcuni. Per comprendere a fondo i concetti legati alla compressione dei dati, una lettura consigliata è il fantastico libro [Understanding Compression: Data Compression for Modern Developers](#).

Codifica VLC:

Supponiamo di avere un flusso di simboli: **a**, **e**, **r** e **t**, con le seguenti probabilità (valori da 0 a 1) di comparire nel flusso.

	a	e	r	t
probabilità	0,3	0,3	0,2	0,2

30

Possiamo assegnare un codice binario (preferibilmente piccolo) al simbolo più probabile, e un codice più lungo ai simboli meno probabili.

	a	e	r	t
probabilità	0,3	0,3	0,2	0,2
codice binario	0	10	110	1110

Proviamo ora a comprimere il flusso rappresentato dalla parola **eat**. Assumendo di utilizzare 8 bit per ogni simbolo (carattere), avremmo bisogno di **24 bit** per rappresentare la parola, senza utilizzare nessun metodo di compressione. Ma se invece utilizziamo i codici binari associati alle lettere, possiamo risparmiare spazio.

Il primo passo è codificare il simbolo **e**, che corrisponde a **10**, per poi passare a **a**, con cui otteniamo **[10][0]** e infine il simbolo **t**, per ottenere **[10][0][1110]**, o **1001110**. Il risultato richiede soltanto **7 bit** per essere memorizzato, 3,4 volte in meno rispetto ai 24 bit iniziali.

È importante notare che ogni codice binario deve avere un prefisso unico (*proprietà del prefisso*). La [codifica di Huffman](#) aiuta a calcolare questi codici. Nonostante questa tecnica abbia qualche problema, ci sono ancora [alcuni codec video che la offrono](#) come opzione.

Sia il codificatore (encoder) che il decodificatore (decoder) **devono conoscere la tabella simboli-codici**, per cui è necessario salvare anche la tabella.

Codifica aritmetica:

Supponiamo di avere il flusso di simboli **a, e, r, s** e **t**, le cui probabilità sono rappresentate dalla tabella.

	a	e	r	s	t
probabilità	0,3	0,3	0,15	0,05	0,2

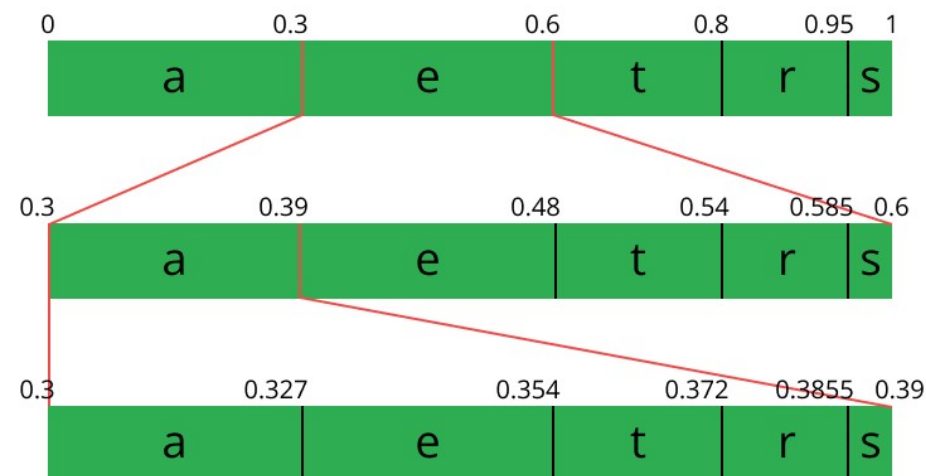
Usando le probabilità possiamo associare degli intervalli ai simboli, ordinandoli per probabilità decrescente.



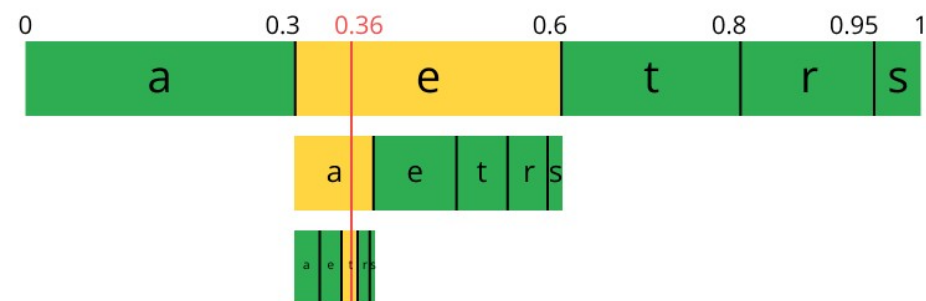
Ora codifichiamo il flusso **eat**: prendiamo il primo simbolo **a**, il cui intervallo va **da 0,3 a 0,6** (non incluso). Lo dividiamo nuovamente, usando le stesse proporzioni della divisione originale, solo che applicate al sottointervallo.



Continuiamo con la codifica del flusso **eat**, prendendo il simbolo **a** che corrisponde ora all'intervallo che va **da 0,3 a 0,39**. Infine ripetiamo il processo con il simbolo **t**, ottenendo l'ultimo intervallo tra **0,354 e 0,372**.



Dobbiamo ora semplicemente prendere un qualsiasi valore contenuto tra **0,354 e 0,372**, ad esempio **0,36**, ma potremmo prenderne uno qualsiasi. Con **solo** questo numero siamo in grado di ricostruire il flusso di simboli originale **eat**. Se ci pensi, è come se stessimo tracciando una riga che attraversa gli intervalli e i sottointervalli per codificare il flusso.



Il **processo inverso** (la decodifica) è ugualmente facile. Avendo a disposizione il numero **0,36** e l'intervallo originale possiamo seguire la stessa procedura, usando il numero per scoprire sottointervallo per sottointervallo il flusso di simboli associato.

In pratica, usando il primo intervallo troviamo che il numero è compreso nell'intervallo del simbolo **e**, per cui prendiamo quell'intervallo e lo dividiamo di nuovo. Ripetiamo quanto appena fatto per trovare che il valore **0,36** è compreso nell'intervallo di **a**, che è quindi il secondo simbolo del flusso. Infine troviamo l'ultimo simbolo **t**, che va a completare il flusso finale **eat**.

Sia l'encoder che il decoder **devono conoscere** la tabella dei simboli con le probabilità, per cui la tabella deve essere salvata e trasmessa.

Interessante vero? Ci devono essere persone incredibilmente intelligenti per riuscire a inventare una soluzione del genere, che oggi viene usata [in molti codec video](#).

L'idea che deve restare è che si può prendere il flusso di bit (bitstream) risultante dalla quantizzazione e comprimerlo in modo lossless, cioè senza perdita di informazioni. Sicuramente questo articolo manca di molti dettagli, spiegazioni, vantaggi e compromessi, ma puoi [imparare facilmente di più se sei uno sviluppatore](#). I nuovi codec stanno iniziando ad usare differenti sistemi di codifica dell'entropia, [come ANS](#).

Esercizio: CABAC vs CAVLC

Puoi [generare due video, uno con codifica dell'entropia CABAC e l'altro con](#) , e [confrontare il tempo di codific](#), così come la [dimensione del file finale](#).

6° passo - formato bitstream

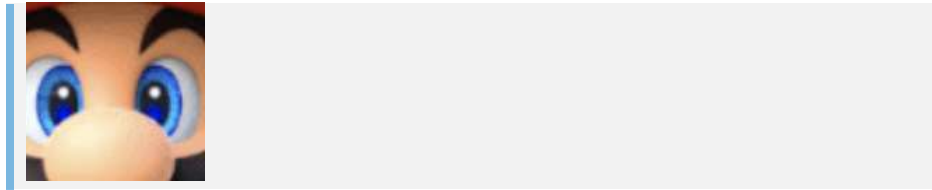
Dopo aver completato tutti questi passaggi, ora dobbiamo **compattare tutti i fotogrammi compressi con le loro informazioni di contest**. Dobbiamo informare in modo esplicito il decoder delle **decisioni che sono state prese dall'encoder durante la codifica**, tra cui la profondità di bit, lo spazio colore, la risoluzione, le predizioni (vettori di moto inter-frame, direzione della predizione intra-frame), il profilo, il livello, il frame rate, il tipo di fotogramma, il numero di fotogramma e molto altro.

Stiamo per studiare, superficialmente, il bitstream del codec H.264. La prima cosa da fare è [generare un bitstream H.264 minimo*](#), usando [ffmpeg](#).

```
./s/ffmpeg -i /files/i/minimal.png -pix_fmt yuv420p /files/v/minimal_yuv420.h264
```

*ffmpeg aggiunge, in modo predefinito, tutti i parametri di codifica come un **NAL SEI**. Presto definiremo che cos'è un NAL.

Il comando genera un bitstream H.264 "raw" (senza formato contenitore), con un **singolo fotogramma**, di dimensione 64x64 pixel, con spazio colore yuv420 (cioè YCbCr 4:2:0) e usando l'immagine seguente come fotogramma.



Bitstream di H.264

Lo standard AVC (H.264) stabilisce che le informazioni vengano trasmesse in **macro frame** (nel significato legato alle reti di telecomunicazioni), chiamati **NAL** (Network Abstraction Layer). L'obiettivo principale dei NAL

è fornire una rappresentazione del video che sia adatta per la trasmissione in rete, considerato che lo standard deve funzionare sulle tv (basate su flussi) e tramite Internet (basato su pacchetti), tra gli altri.



Viene usato un **segnale di sincronizzazione** (*synchronization marker*) per delimitare le unità NAL. Ogni segnale di sincronizzazione corrisponde al valore fisso `0x00 0x00 0x01`, ad eccezione del primo che ha il valore `0x00 0x00 0x00 0x01`. Se eseguiamo il comando **hexdump** sul bitstream H.264 che abbiamo generato, riusciamo facilmente ad identificare almeno tre unità NAL all'inizio del file.

```
$ hexdump v/minimal_yuv420.h264
00000000 00 00 00 01 67 64 00 0a ac d9 44 26 84 00 00 03
00000010 00 04 00 00 03 00 c8 3c 48 96 58 00 00 00 01 68
00000020 eb e3 cb 22 c0 00 00 01 06 05 ff ff 9c dc 45 e9
00000030 bd e6 d9 48 b7 96 2c d8 20 d9 23 ee ef 78 32 36
00000040 34 20 2d 20 63 6f 72 65 20 31 34 38 20 2d 20 48
00000050 2e 32 36 34 2f 4d 50 45 47 2d 34 20 41 56 43 20
00000060 63 6f 64 65 63 20 2d 20 43 6f 70 79 6c 65 66 74
00000070 20 32 30 30 33 2d 32 30 31 36 20 2d 20 68 74 74
00000080 70 3a 2f 2f 77 77 77 2e 76 69 64 65 6f 6c 61 6e
00000090 2e 6f 72 67 2f 78 32 36 34 2e 68 74 6d 6c 20 2d
000000a0 20 6f 70 74 69 6f 6e 73 3a 20 63 61 62 61 63 3d
000000b0 31 20 72 65 66 3d 33 20 64 65 62 6c 6f 63 6b 3d
```

Come detto prima, il decodificatore ha bisogno non solo dei dati dell'immagine, ma anche di dettagli come il numero di fotogramma, i colori, i parametri usati e altro. Il **primo byte** di ciascun NAL definisce la sua categoria e il **tipo**.

ID del tipo di NAL	Descrizione
0	Non definito
1	Slice codificato di un'immagine non IDR
2	Slice codificato di una partizione A
3	Slice codificato di una partizione B
4	Slice codificato di una partizione C
5	IDR Slice codificato di un'immagine IDR
6	SEI Informazioni aggiuntive (<i>supplemental enhancement information</i>)
7	SPS Insieme parametri di sequenza (<i>sequence parameter set</i>)
8	PPS Insieme parametri d'immagine (<i>picture parameter set</i>)
9	Delimitatore di unità di accesso
10	Fine della sequenza
11	Fine del flusso

ID del tipo di NAL	Descrizione
...	...

Solitamente il primo NAL di un bitstream è di tipo **SPS**. Questo tipo di NAL contiene informazioni generiche sulle variabili di codifica, come il **profi**, il **livello**, la **risoluzione** e altro.

Se saltiamo il primo segnale di sincronizzazione possiamo decodificare il **primo byte** per capire il **tipo del primo NAL** del bitstream di esempio.

Ad esempio in questo caso il primo byte dopo la sincronizzazione è **01100111**, dove il primo bit (0) si riferisce al campo **forbidden_zero_bit**, i due bit successivi (11) indicano il campo **nal_ref_idc**, che stabilisce se il NAL è un campo di riferimento oppure no, e infine gli ultimi cinque bit (01111) sono il campo **nal_unit_type**, cioè il tipo di NAL, in questo caso **SPS** (7).

Il secondo byte (**bin=01100100**, **hex=0x64**, **dec=100**) di un NAL SPS è il campo **profile_idc**, che indica il profilo che è stato usato per la codifica. In questo esempio è stato usato il profilo "**Constrained High**", che è un profilo "alto" senza il supporto agli slice di tipo B (bi-predittive).

```
$ xxd -b -c 11 v/minimal_yuv420.h264
00000000: 00000000 00000000 00000000 00000001 01100111 01100100 00000000 00001010 10101100 11011001 01000100
0000000b: 00100110 10000100 00000000 00000000 00000011 00000000 00000100 00000000 00000000 00000011 00000000
00000016: 11001000 00111100 01001000 10010110 01011000 00000000 00000000 00000001 01101000 11101011
00000021: 11100011 11001011 00100010 11000000 00000000 00000000 00000001 00000110 00000101 11111111 11111111
0000002c: 10011100 11011100 01000101 11101001 10111101 11100110 11011001 01001000 10110111 10010110 00101100
00000037: 11011000 00100000 11011001 00100011 11101110 11101111 01111000 00110010 00110110 00110100 00100000
00000042: 00101101 00100000 01100011 01101111 01110010 01100101 00100000 00110001 00110100 00111000 00100000
0000004d: 00101101 00100000 01001000 00101110 00110010 00110110 00110100 00101111 01001101 01010000 01000101
```

Se leggiamo le specifiche del bitstream H.264 troveremo diversi valori per i campi **nome paramet**, **categoria** e **descrizione**. Ad esempio consideriamo i campi **pic_width_in_mbs_minus_1** e **pic_height_in_map_units_minus_1**.

Nome parametro	Categoria	Descrizion
pic_width_in_mbs_minus_1	0	ue
pic_height_in_map_units_minus_1	0	ue(v)

ue(v): intero senza segno **codificato con Exp-Golomb**

Facendo un po' di calcoli possiamo utilizzare il valore di questi campi per ricavare la **risoluzione**. Ad esempio una risoluzione di **1920 x 1080** può essere ottenuta dando al parametro **pic_width_in_mbs_minus_1** il valore di **119** ($(119 + 1) * dimensione_macroblocco = 120 * 16 = 1920$). Abbiamo risparmiato spazio, memorizzando **119** anziché **1920**.

Se continuiamo a esaminare il video creato con un visualizzatore binario (es. **xxd -b -c 11 v/minimal_yuv420.h264**), possiamo saltare all'ultimo NAL che è il fotogramma vero e proprio.

```
000002c0: 00110001 00111010 00110001 00101110 00110000 00110000 00000000 10000000 00000000 00000000 00000001
000002cb: 01100101 10001000 10000100 00000000 00100001 11111111 01000001 01001010 00110110 11100110 01111000
000002d6: 00101101 10000101 10011011 01001010 10100101 11110001 00111010 11100110 11000111 10111100 10000010
000002e1: 10110010 01101100 10010011 11000001 10101010 11001010 01011111 10100010 10100011 10001011 10011111
```

Il valore dei primi 6 byte è **01100101 10001000 10000100 00000000 00100001 11111111**. Come abbiamo visto dal primo byte si può ricavare il tipo di NAL, in questo caso **00101**, che corrisponde a **Slice IDR (5)**. Continuiamo con l'ispezione.

7.3.3 Slice header syntax

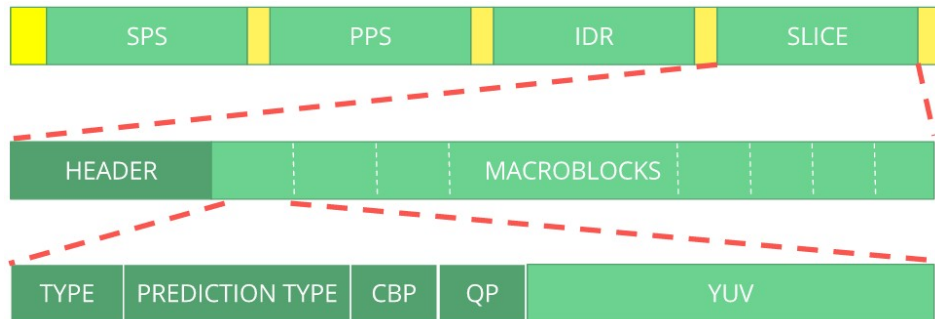
slice_header() {	C	Descriptor
first_mb_in_slice	2	ue(v)
slice_type	2	ue(v)
pic_parameter_set_id	2	ue(v)
if(separate_colour_plane_flag == 1)		
colour_plane_id	2	u(2)
frame_num	2	u(v)
if(!frame_mbs_only_flag) {		
field_pic_flag	2	u(1)
if(field_pic_flag)		
bottom_field_flag	2	u(1)
}		
if(!IdrPicFlag)		
idr_pic_id	2	ue(v)
if(pic_order_cnt_type == 0) {		
pic_order_cnt_lsb	2	u(v)
if(bottom_field_pic_order_in_frame_present_flag && !field_pic_flag)		
delta_pic_order_cnt_bottom	2	se(v)
}		
if(pic_order_cnt_type == 1 && !delta_pic_order_always_zero_flag) {		
delta_pic_order_cnt[0]	2	se(v)

Utilizzando le specifiche dell'header degli slice H.264 possiamo decodificare il tipo di slice (**slice_type**), il numero del fotogramma (**frame_num**), assieme ad altre importanti informazioni.

Per ottenere il reale valore di alcuni camp **ue(v)**, **me(v)**, **se(v)**, **te(v)**) dobbiamo decodificarli utilizzando un metodo speciale chiamato **Exponential-Golomb**, che è un **modo molto efficiente per codificare valori variabili**, specialmente quando vengono usati molti valori predefiniti.

I valori **slice_type** e **frame_num** sono 7 (slice di tipo I) e 0 (primo fotogramma).

Possiamo vedere il **bitstream come un protocollo**, e se vuoi o devi imparare di più a proposito del bitstream fai riferimento alle **specifiche H.264 dell'ITU**. Ecco un diagramma semplificato che mostra dove vengono memorizzati i dati dell'immagine vera e propria (YCbCr/YUV compresso).



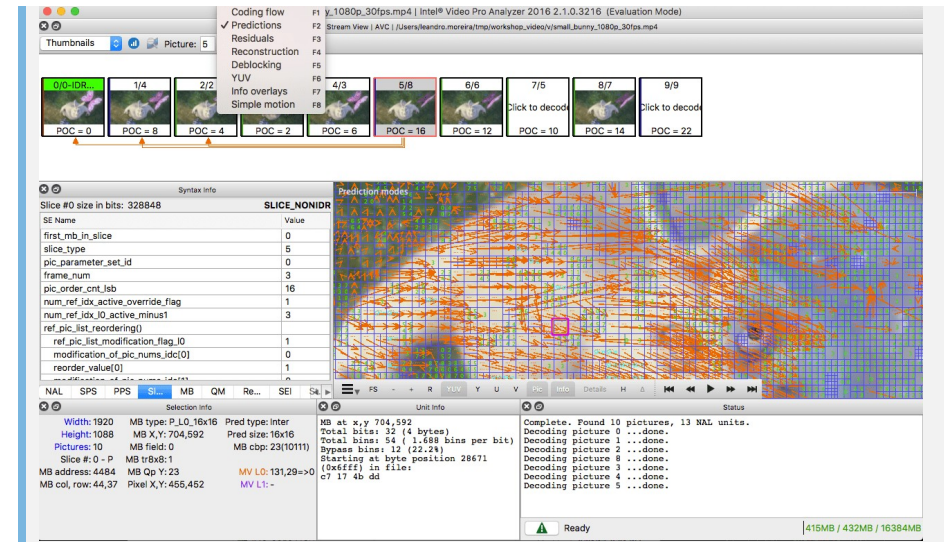
Possiamo esplorare altri bitstream come quello di VP9, H.265 (HEVC) o anche il nostro **nuovo miglior amico AV1**. Si assomigliano? No, ma una volta imparato uno capire gli altri è facile.

Esercizio: ispezionare il bitstream H.264

Possiamo [generare un video con un singolo fotogramma](#) e usare [mediainfo](#) per ispezionare il suo bitstream H.264. Infatti puoi anche vedere il [codice sorgente che si occupa del parsing del bitstream H.264 \(AV\)](#).

```
000002D4 slice_layer_without_partitioning (IDR) - 0 - Frame 0 - slice_type I - frame_num 0 - pic_order_cnt_lsb 0 (551 bytes)
000002D4 Header (5 bytes)
000002D4 size: 547 (0x00000223)
000002D0 nal_ref_idc: 3 (0x03)(2 bits) -
000002D0 nal_unit_type: 5 (0x05)(5 bits) -
000002D9 slice_header (4 bytes)
000002E0 first_mb_in_slice: 0 (0x0)
000002D9 slice_type: 7 (0x7)I -
000002E0 pic_parameter_set_id: 0 (0x0)
000002DC frame_num: 0 (0x0)
000002D8 idr_pic_id: 0 (0x0)
000002D0 pic_order_cnt_lsb: 0 (0x0)
000002DC no_output_of_prior_pics_flag: No
000002D8 long_term_reference_flag: No
000002DA slice_qp_delta: 8 (0x8)
000002D9 disable_deblocking_filter_idc: 0 (0x0)
000002E0 slice_alpha_c0_offset_div2: 0 (0x0)
000002DF slice_beta_offset_div2: 0 (0x0)
000002DF slice_data (540 bytes)
000002DF (ToDo): (Data)
```

In alternativa possiamo usare il software [Intel Video Pro Analyzer](#) (è a pagamento ma esiste una versione di prova che limita l'analisi ai primi 10 fotogrammi).



Ripasso

Molti dei **codec moderni utilizzano lo stesso modello di codifica che abbiamo imparato**. Ad esempio, diamo un'occhiata al diagramma a blocchi del codec video Thor, che contiene tutti i passi che abbiamo studiato. A questo punto dovresti essere in grado almeno di comprendere meglio le innovazioni e gli articoli che riguardano questi argomenti.

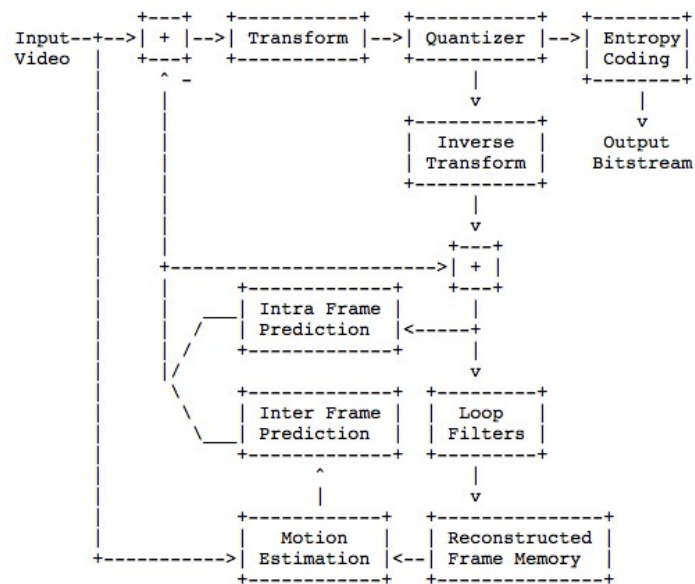


Figure 1: Encoder Structure

Inizialmente abbiamo calcolato che ci servono **139 GB di spazio per archiviare un file di un'ora nel formato 720p30**. Se applichiamo le tecniche imparate qui, come **la predizione intra-frame e inter-frame, la trasformata, la quantizzazione, la codifica dell'entropia**, assumendo che spendiamo **0,031 bit per pixel** possiamo raggiungere la stessa qualità percepita utilizzando soltanto **368 MB anziché 139 GB**.

Abbiamo scelto di usare **0,031 bit per pixel** basandoci sull'esempio precedente.

Come fa H.265 a comprimere più di H.264?

Ora che sappiamo come funziona un codec, risulta più facile capire come i nuovi codec sono in grado di memorizzare risoluzioni più alte usando meno bit.

Confronteremo AVC e HEVC, tenendo in mente che il processo è quasi sempre un compromesso tra la complessità dell'algoritmo (cicli di CPU) e il tasso di compressione.

HEVC ha **parti** (e **sottopartizioni**) più grandi e più numerose rispetto ad AVC, inoltre **più direzioni di predizione intra-frame**, una **migliore codifica dell'entropia**, e altro. Tutto insieme fa in modo che H.265 sia in grado di comprimere il 50% in più rispetto ad H.264.

H.264/AVC

16x16 Macroblock

Block coding structure

3 Intra partitioning

4 Inter partitioning

+4 sub-partitioning 8x8

2 Transform sizes:
4x4, 8x8

Up to 9 Intra prediction directions

HEVC

Coding Unit
64x64 to 8x8

Quadtree coding structure

Prediction Unit and Transform Unit partitioning

⇒ Multiples sizes/forms: 64x64 to 4x4

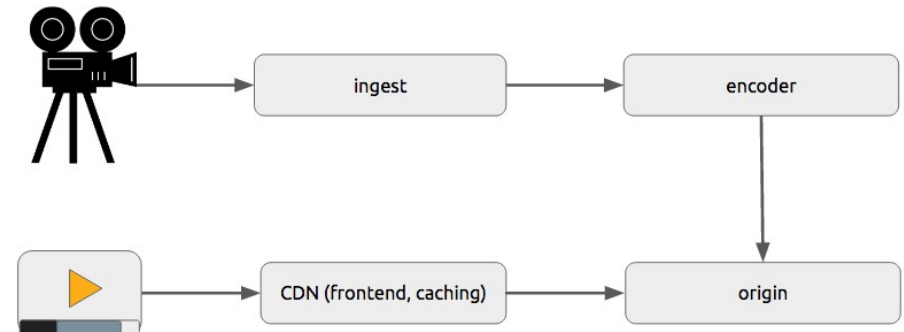
35 Intra prediction directions

Efficient spatio-temporal mv prediction

<http://www.worldbroadcastingunions.org/wbuarea/library/docs/isog/presentations/20128/2.4%20Vieron%20ATEME.pdf>

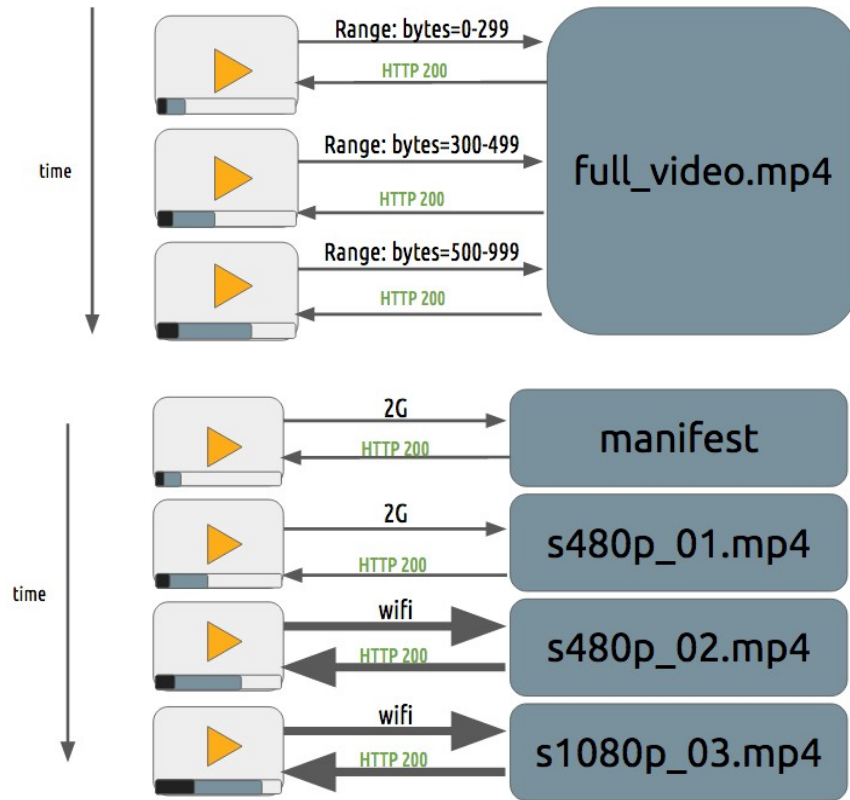
Streaming onl

Architettura generale



[TODO]

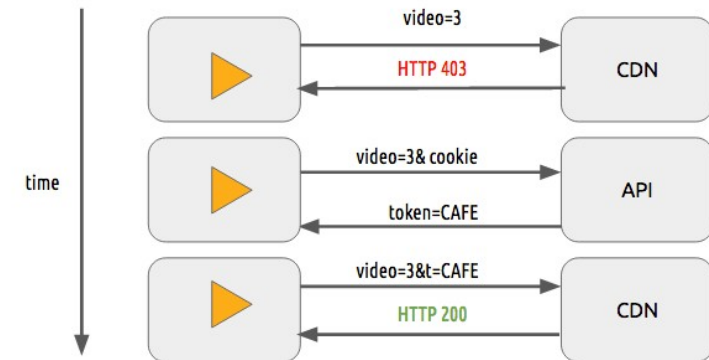
Download progressivo vs streaming adattivo



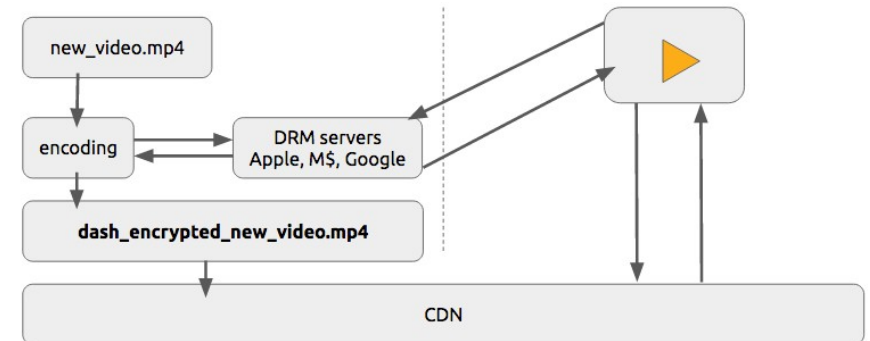
[TODO]

Protezione dei contenuti

Possiamo usare un semplice **sistema a token** (gettone) per proteggere i contenuti. Un utente senza token che prova a richiedere un video viene bloccato dalla CDN, mentre un utente con un token valido può riprodurre il contenuto. Funziona in modo abbastanza simile alla maggior parte dei sistemi di autenticazione sul web.



L'utilizzo di questo sistema consente comunque all'utente di scaricare e ridistribuire il video. Un sistema **DRM (digital rights management)** permette di evitare anche questo problema.



I sistemi di produzione nel mondo reale in genere usano entrambe le tecniche, per offrire sia autorizzazione che autenticazione.

DRM

Sistemi principali

- FPS - **FairPlay Streaming**
- PR - **PlayReady**
- WV - **Widevine**

Cos'è?

DRM sta per *gestione dei diritti digitali (Digital Rights Management)* ed è un modo per **aggiungere una protezione del diritto d'autore ai contenuti digitali**, quindi anche al video e all'audio. Nonostante sia molto usato, **non è universalmente accettato**.

Perché?

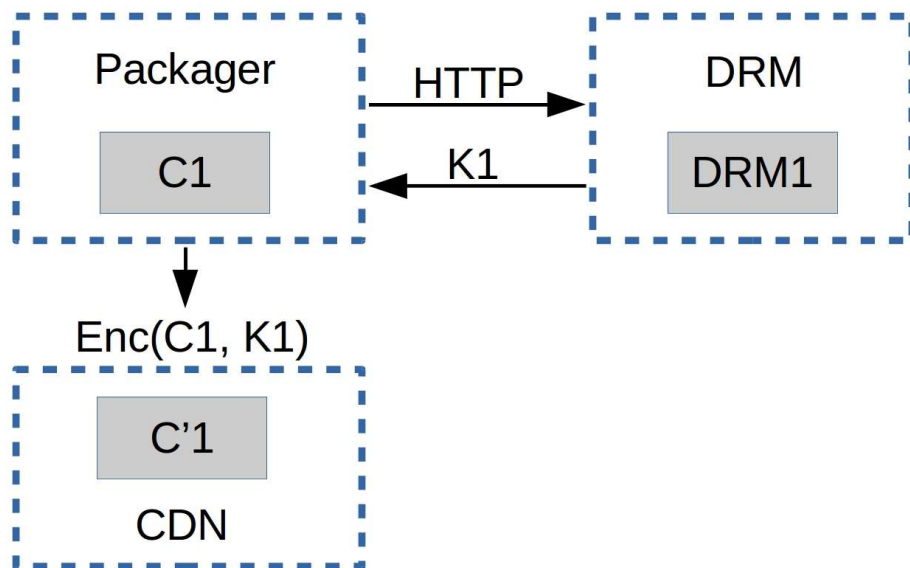
I creatori di contenuti (principalmente gli studi) vogliono proteggere la proprietà intellettuale dalla copia, per prevenire la ridistribuzione non autorizzata dei contenuti digitali.

Come?

Descriveremo un modello generico ed astratto di DRM in un modo molto semplificato.

Sia dato un **contenuto C1** (es. uno streaming video HLS o DASH), con un **player P1** (es. shaka-clappr, exoplayer o iOS) e un **dispositivo D1** (es. uno smartphone, una tv, un tablet o un computer), che utilizzano un **sistema DRM che chiamiamo DRM1** (es. Widevine, PlayReady, FairPlay).

Il contenuto C1 viene cifrato con una **chiave simmetrica K1** fornita dal sistema DRM1, così da ottenere il **contenuto cifrato C'1**.



Il player P1 del dispositivo D1 possiede due chiavi (asimmetriche), cioè una **chiave privata PRK1** (questa chiave è protetta¹ e conosciuta solo da **D1**) e una **chiave pubblica PUK1**.

¹**protetta**: questa protezione può essere ottenuta sia **tramite hardware**, memorizzando la chiave all'interno di un chip speciale in sola lettura che agisce come una **scatola nera** per fornire la decifratura, sia **tramite softw** (in modo meno sicuro). Il sistema DRM offre dei modi per sapere quale tipo di protezione il dispositivo ha a disposizione.

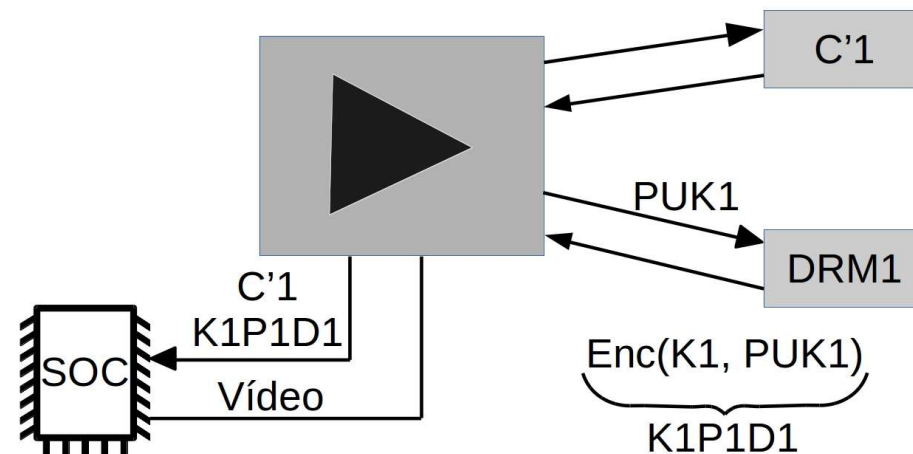
Quando il **player P1 vuole riprodurre il contenuto C'1**, deve contrattare con il **sistema DRM1**, fornendogli la sua chiave pubblica **PUK1**. Il sistema DRM1 ritorna la **chiave K1 cifrata** con la chiave pubblica **PUK1** del client. A livello teorico, questa risposta è qualcosa che **soltanto D1 è in grado di decifrare**.

$$K1P1D1 = \text{enc}(K1, \text{PUK1})$$

Il player **P1** utilizza il suo sistema DRM locale (che potrebbe essere un **SoC**, una parte di software o hardware specializzata), **in grado di decifrare** il contenuto utilizzando la sua chiave privata **PRK1**. In questo modo può **decifrare la chiave simmetrica K1 da K1P1D1** e quindi riprodurre **C'1**. Nel migliore dei casi, le chiavi non sono esposte nella memoria RAM.

```
K1 = dec(K1P1D1, PRK1)
```

```
P1.play(dec(C'1, K1))
```



Come usare jupyter

Assicurati di avere **Docker installato** e avvia `./s/start_jupyter.sh`. Segui poi le istruzioni mostrate nel terminale.

Conferenze

- [DEMUXED](#) - puoi [guardare le presentazioni degli ultimi 2 eventi](#).

Riferimenti

Il contenuto più ricco è qua, puoi trovare le informazioni che hanno ispirato questo testo e da cui sono stati estratti i concetti. Puoi approfondire la tua conoscenza leggendo questi link, libri, video, ecc.

Corsi online e tutorial:

- <https://www.coursera.org/learn/digital/>
- https://people.xiph.org/~tteribe/pubs/lca2012/auckland/intro_to_video1.pdf
- <https://xiph.org/video/vid1.shtml>
- <https://xiph.org/video/vid2.shtml>
- <http://slhck.info/ffmpeg-encoding-course>
- <http://www.cambridgeincolour.com/tutorials/camera-sensors.htm>
- <http://www.slideshare.net/vcodex/a-short-history-of-video-coding>
- <http://www.slideshare.net/vcodex/introduction-to-video-compression-13394338>
- <https://developer.android.com/guide/topics/media/media-formats.html>
- <http://www.slideshare.net/MadhawaKasun/audio-compression-23398426>
- http://inst.eecs.berkeley.edu/~ee290t/sp04/lectures/02-Motion_Compensation_girod.pdf

Libri:

- https://www.amazon.com/Understanding-Compression-Data-Modern-Developers/dp/1491961538/ref=sr_1_1?s=books&ie=UTF8&qid=1486395327&sr=1-1
- <https://www.amazon.com/H-264-Advanced-Video-Compression-Standard/dp/0470516925>
- https://www.amazon.com/Practical-Guide-Video-Audio-Compression/dp/0240806301/ref=sr_1_3?s=books&ie=UTF8&qid=1486396914&sr=1-3&keywords=A+PRACTICAL+GUIDE+TO+VIDEO+AUDIO
- https://www.amazon.com/Video-Encoding-Numbers-Eliminate-Guesswork/dp/0998453005/ref=sr_1_1?s=books&ie=UTF8&qid=1486396940&sr=1-1&keywords=jan+ozer

Specifiche bitstream:

- <http://www.itu.int/rec/T-REC-H.264-201610-I>
- <http://www.itu.int/ITU-T/recommendations/rec.aspx?rec=12904&lang=en>
- <https://storage.googleapis.com/downloads.webmproject.org/docs/vp9/vp9-bitstream-specification-v0.6-20160331-draft.pdf>
- http://iphone.hhi.de/wiegand/assets/pdfs/2012_12_IEEE-HEVC-Overview.pdf
- http://phenix.int-evry.fr/jct/doc_end_user/current_document.php?id=7243
- <http://gentlelogic.blogspot.com.br/2011/11/exploring-h264-part-2-h264-bitstream.html>
- <https://forum.doom9.org/showthread.php?t=167081>
- <https://forum.doom9.org/showthread.php?t=168947>

Software:

- <https://ffmpeg.org/>
- <https://ffmpeg.org/ffmpeg-all.html>
- <https://ffmpeg.org/ffprobe.html>
- <https://trac.ffmpeg.org/wiki/>
- <https://software.intel.com/en-us/intel-video-pro-analyzer>
- <https://medium.com/@mbebenita/av1-bitstream-analyzer-d25f1c27072b#.d5a890xx8>

Codec non ITU:

- <https://aomedia.googlesource.com/>
- <https://github.com/webmproject/libvpx/tree/master/vp9>
- <https://people.xiph.org/~xiphmont/demo/daala/demo1.shtml>
- <https://people.xiph.org/~jm/daala/revisiting/>

- <https://www.youtube.com/watch?v=lzPaldsmJbk>
- https://fosdem.org/2017/schedule/event/om_av1/
- https://jmvalin.ca/papers/AV1_tools.pdf

Concetti di codifica:

- <http://x265.org/hevc-h265/>
- <http://slhck.info/video/2017/03/01/rate-control.html>
- <http://slhck.info/video/2017/02/24/vbr-settings.html>
- <http://slhck.info/video/2017/02/24/crf-guide.html>
- <https://arxiv.org/pdf/1702.00817v1.pdf>
- <https://trac.ffmpeg.org/wiki/Debug/MacroblocksAndMotionVectors>
- <http://web.ece.ucdavis.edu/cerl/ReliableJPEG/Cung/jpeg.html>
- http://www.adobe.com/devnet/adobe-media-server/articles/h264_encoding.html
- <https://prezi.com/8m7thtvl4ywr/mp3-and-aac-explained/>
- <https://blogs.gnome.org/rbultje/2016/12/13/overview-of-the-vp9-video-codec/>
- <https://videoblerg.wordpress.com/2017/11/10/ffmpeg-and-how-to-use-it-wrong/>

Sequenze video per i test:

- <http://bbb3d.renderfarming.net/download.html>
- <https://www.its.bldrdoc.gov/vqeg/video-datasets-and-organizations.aspx>

Var

- <http://stackoverflow.com/a/24890903>
- <http://stackoverflow.com/questions/38094302/how-to-understand-header-of-h264>
- <http://techblog.netflix.com/2016/08/a-large-scale-comparison-of-x264-x265.html>
- <http://vanseodesign.com/web-design/color-luminance/>
- <http://www.biologymad.com/nervoussystem/eyenotes.htm>
- http://www.compression.ru/video/codecs_comparison/h264_2012/mpeg4_avc_h264_video_codecs_comparison.pdf
- <http://www.csc.villanova.edu/~rschumey/csc4800/dct.html>
- <http://www.explainthatstuff.com/digitalcameras.html>
- <http://www.hkvstar.com>
- <http://www.hometheatersound.com/>
- <http://www.lighterra.com/papers/videoencodingh264/>
- <http://www.red.com/learn/red-101/video-chroma-subsampling>
- <http://www.slideshare.net/ManoharKuse/hevc-intra-coding>
- <http://www.slideshare.net/mwalendo/h264vs-hevc>
- <http://www.slideshare.net/rvarun7777/final-seminar-46117193>
- http://www.springer.com/cda/content/document/cda_downloadaddocument/9783642147029-c1.pdf
- <http://www.streamingmedia.com/Articles/Editorial/Featured-Articles/A-Progress-Report-The-Alliance-for-Open-Media-and-the-AV1-Codec-110383.aspx>
- <http://www.streamingmediaglobal.com/Articles/ReadArticle.aspx?ArticleID=116505&PageNum=1>
- <http://yumichan.net/video-processing/video-compression/introduction-to-h264-nal-unit/>
- <https://cardinalpeak.com/blog/the-h-264-sequence-parameter-set/>
- <https://cardinalpeak.com/blog/worlds-smallest-h-264-encoder/>

- <https://codesequoia.wordpress.com/category/video/>
- https://developer.apple.com/library/content/technotes/tn2224/_index.html
- https://en.wikibooks.org/wiki/MeGUI/x264_Settings
- https://en.wikipedia.org/wiki/Adaptive_bitrate_streaming
- https://en.wikipedia.org/wiki/AOMedia_Video_1
- https://en.wikipedia.org/wiki/Chroma_subsampling#/media/File:Colorcomp.jpg
- https://en.wikipedia.org/wiki/Cone_cell
- https://en.wikipedia.org/wiki/File:H.264_block_diagram_with_quality_score.jpg
- https://en.wikipedia.org/wiki/Inter_frame
- https://en.wikipedia.org/wiki/Intra-frame_coding
- https://en.wikipedia.org/wiki/Photoreceptor_cell
- https://en.wikipedia.org/wiki/Pixel_aspect_ratio
- https://en.wikipedia.org/wiki/Presentation_timestamp
- https://en.wikipedia.org/wiki/Rod_cell
- https://it.wikipedia.org/wiki/File:Pixel_geometry_01_Pengo.jpg
- <https://leandromoreira.com.br/2016/10/09/how-to-measure-video-quality-perception/>
- <https://sites.google.com/site/linuxencoding/x264-ffmpeg-mapping>
- <https://softwaredevelopmentperestroika.wordpress.com/2014/02/11/image-processing-with-python-numpy-scipy-image-convolution/>
- <https://tools.ietf.org/html/draft-fuldseth-netvc-thor-03>
- <https://www.encoding.com/android/>
- <https://www.encoding.com/http-live-streaming-hls/>
- <https://web.archive.org/web/20150129171151/https://www.iem.thm.de/telekom-labor/zinke/mk/mpeg2beg/whatisit.htm>
- <https://www.lifewire.com/cmos-image-sensor-493271>
- <https://www.linkedin.com/pulse/brief-history-video-codecs-yoav-nativ>
- <https://www.linkedin.com/pulse/video-streaming-methodology-reema-majumdar>
- <https://www.vcodex.com/h264avc-intra-precision/>
- <https://www.youtube.com/watch?v=9vgtJJ2wwMA>
- <https://www.youtube.com/watch?v=LFxN9PiOGtY>
- https://www.youtube.com/watch?v=Lto-ajuqW3w&list=PLzH6n4zXuckpKAj1_88VS-8Z6yn9zX_P6
- <https://www.youtube.com/watch?v=LWxu4rkZBL>
- https://web.stanford.edu/class/ee398a/handouts/lectures/EE398a_MotionEstimation_2012.pdf